



**UNIVERSITÀ
DI PADOVA**



Corso di Laurea Triennale in Ingegneria Informatica

Controllo della temperatura in un dispositivo TCLab mediante un algoritmo di tipo PILCO-RL

Relatore:

Prof. Rampazzo Mirco

Laureando/a:

Alberto Bortoletto

Matricola:

2101761

Anno Accademico 2025/2026

Grazie.

Il primo, più grande grazie va alla mia famiglia, che ha sempre creduto in me e mi ha sostenuto in ogni modo possibile: senza di voi non sarei arrivato fin qui, e mi sento davvero fortunato, vi voglio un mondo di bene.
A mio fratello Giacomo che mi ha sempre spinto a dare il massimo,
alle nonne: Anna, Tina,
ai nonni che non ci sono più: Gino, Dante
a mia zia Melania per ricordarmi di andarla a trovare ogni tanto,
a Nicole, esempio di perseveranza,
ai miei amici e colleghi Mattia, Morris, Jack, Stefano, Sergio, Fede, Elena e David, che mi hanno accompagnato in questi 3 anni di impegno,
al gruppo delle coddate — Enrico, Michi Steva, Michi Sadocco e il Daddy — che mi ricordano di uscire ogni tanto,
ad Ale Del, uno degli amici più fedeli che abbia mai avuto,
al gruppo di palestra,
a Letizia, che mi ha fatto da sostegno morale durante l'università,
a Fil che mi ha sempre appoggiato,
a Valerik presente nelle scelte difficili,
a Scabbia e Pete, per le intere giornate di studio passate su Discord.

Il grazie più importante va a Chiara, la mia ragazza. Una persona fondamentale nella mia vita: rendi ogni momento indimenticabile, mi lasci la spensieratezza e la serenità di cui ho bisogno, specialmente in questi duri mesi di lezioni, esami e tesi. Non basterebbe l'intera tesi per descrivere l'effetto che mi fai. Oggi stai inseguendo il tuo sogno al corso dei Carabinieri e non potrai essere fisicamente al mio fianco in questo giorno, ma per me è come se ci fossi: ti ho sentita vicina in ogni momento, e questo traguardo è anche tuo.

Sono fiera di te.

Ti amo, piccola mia.

E infine un grazie alla persona più importante di sempre: me stesso.

Grazie per non aver mai mollato, per aver continuato a credere in questo obiettivo anche quando sembrava lontano, e per non esserti lasciato abbattere da chi ha provato a metterti i bastoni tra le ruote. Certe persone le hai lasciate indietro lungo la strada, ed è stato giusto così: probabilmente volevano essere qui, ma quello che conta è che tu abbia vinto con le persone a cui tieni.

*”Lo abbiamo fatto, davvero, davvero,
anche se non ci prendevi sul serio.”*

Sommario

Questo elaborato presenta l'implementazione e la valutazione di PILCO (*Probabilistic Inference for Learning COntrol*), un algoritmo di *policy search* model-based proposto da Deisenroth e Rasmussen (2011), applicato al controllo termico di precisione sul sistema TCLAB (*Temperature Control Lab*).

L'obiettivo principale è dimostrare come l'uso di un modello probabilistico della dinamica — basato su *Gaussian Processes* (GP) — consenta di raggiungere prestazioni di controllo soddisfacenti con un numero estremamente ridotto di interazioni con il sistema fisico reale, superando le limitazioni di efficienza campionaria tipiche degli approcci *model-free*.

Inizialmente introdurremo il concetto di *Reinforcement Learning* (RL) applicato al mondo dei controllori, evidenziandone eventuali problemi che vi possono essere e come PILCO riesca a risolverli. Successivamente ci cimenteremo sull'analisi di funzionamento dell'algoritmo con tutti gli aspetti matematici del caso e infine parleremo dell'applicazione PILCO sul sistema termico TCLAB (*Temperature Control Lab*).

Verranno analizzati tre scenari sperimentali: condizioni nominali, variazione della temperatura ambiente e inseguimento di setpoint dinamico in presenza di disturbi. I risultati evidenziano la capacità di PILCO di apprendere un controllore efficiente in pochi *rollout*, gestendo l'incertezza del modello in modo principiato attraverso la propagazione analitica dei momenti. Inoltre per avere un metodo di paragone utilizzeremo il controllore più semplice possibile (*isteresi*) per capirne i vantaggi e gli svantaggi dell'utilizzare un approccio rispetto che ad un altro.

Indice

Sommario	v
1 Introduzione	1
1.1 Contesto: l'ascesa del Reinforcement Learning nei controlli automatici	1
1.2 Il problema della <i>sample efficiency</i>	1
1.3 Obiettivo della tesi	2
1.4 Struttura dell'elaborato	2
2 Stato dell'Arte e Fondamenti Teorici	5
2.1 Sistemi di controllo e Reinforcement Learning	5
2.1.1 Controlli classici: isteresi, PID e MPC	5
2.2 Formalizzazione del Problema di Reinforcement Learning	6
2.2.1 Processi Decisionali di Markov	6
2.2.2 Politica e Ritorno Atteso	7
2.2.3 Approcci Model-Free e Model-Based	7
2.2.4 Il Problema del Model Bias	7
2.3 Gaussian Processes	8
2.3.1 Definizione e Intuizione	8
2.3.2 Il Kernel e la sua Interpretazione	9
2.3.3 Predizione con GP	9
2.4 L'algoritmo PILCO	10
2.4.1 Modello Probabilistico della Dinamica	11
2.4.2 Valutazione Analitica della Politica	11
2.4.3 Funzione di Costo	13
2.4.4 Miglioramento Analitico della Politica	14
2.4.5 Pseudocodice	15
3 Il Sistema Sperimentale: TCLab	17
3.1 Descrizione dell'Hardware	17

3.2	Modellistica Matematica	18
3.2.1	Ipotesi di Modellazione	18
3.2.2	Modello Nonlineare Completo	19
3.2.3	Semplificazione SISO	19
3.2.4	Linearizzazione e Limiti del Modello Lineare	20
3.3	Perché TCLab per PILCO	21
4	Implementazione di PILCO sul TCLab	23
4.1	Struttura del codice	23
4.2	Definizione dello stato e dell'azione	24
4.3	Funzione di costo	24
4.4	Architettura della politica	25
4.5	Modello GP della dinamica	25
4.6	Dinamica ODE del sistema	26
4.7	Loop di addestramento	26
4.8	Parametri dell'esperimento	27
5	Risultati Sperimentali e Confronto con Isteresi	29
5.1	Caso 1 — Regolazione a Setpoint Fisso	29
5.1.1	Setup sperimentale	29
5.1.2	Risultati	30
5.2	Caso 2 — Robustezza alla Temperatura Ambiente	31
5.2.1	Setup sperimentale	31
5.2.2	Risultati	32
5.3	Caso 3 — Inseguimento del Riferimento Variabile	34
5.3.1	Setup sperimentale	34
5.3.2	Risultati	34
5.4	Confronto PILCO vs Controllore a Isteresi	36
5.4.1	Configurazione del confronto	36
5.4.2	Risultati del confronto	37
6	Discussione dei Risultati	39
6.1	Efficienza di campionamento	39
6.2	Interpretazione dell'incertezza	39
6.3	Analisi del confronto PILCO vs isteresi	40
6.3.1	Decisione locale vs decisione globale	40
6.3.2	Apprendimento senza conoscenza a priori	40

6.3.3	Considerazioni sulla comparabilità	41
6.3.4	Confronto dettagliato	41
6.4	Limiti riscontrati	42
7	Conclusioni e Sviluppi Futuri	43
7.1	Sintesi dei risultati	43
7.2	Sviluppi futuri	44
	Bibliografia	45

Elenco delle figure

2.1	Illustrazione del model bias [2]. A sinistra: dataset di transizioni osservate. Al centro: possibili funzioni deterministiche compatibili con i dati — ciascuna produce predizioni arbitrarie fuori dalla zona di training con piena confidenza. A destra: il modello probabilistico GP esprime correttamente l'incertezza nelle zone inesplorate.	8
2.2	Predizione GP con input incerto [2]. La distribuzione di input $p(x_{t-1}, u_{t-1})$ gaussiana (pannello in basso a destra) viene propagata attraverso il modello GP, producendo la distribuzione $p(\delta_t)$ approssimata tramite moment matching (pannello in alto a sinistra).	12
3.1	Il sistema TCLAB: shield Arduino con i due transistor riscaldatori TIP31C (Q_1 , Q_2) e i relativi termistori TMP36GZ (T_1 , T_2) [13].	18
3.2	Confronto tra modello nonlineare e lineare [14]. Sinistra: Q_1 vicino al punto di linearizzazione — il modello lineare è accurato. Destra: Q_1 lontano dal punto di linearizzazione — il modello lineare sovrastima significativamente la temperatura a regime.	20
5.1	Caso 1 — Evoluzione del training: traiettorie $T_1(t)$ e costo cumulativo per i rollout casuali (grigio) e le iterazioni PILCO (colore).	30
5.2	Caso 1 — Ultimo rollout PILCO: profilo $T_1(t)$, azione $Q_1(t)$, costo per step e errore di inseguimento.	31
5.3	Caso 2 — Evoluzione del training sulle quattro temperature ambiente.	32
5.4	Caso 2 — Valutazione: pannello superiore con $T_1(t)$ per ogni T_{amb} di test, segnale di controllo Q_1 e disturbo Q_2 , errore di tracking e costo lossSat sull'intero episodio cucito.	33
5.5	Caso 3 — Scalinata di valutazione: profilo $T_1(t)$ vs riferimento, errore $e(t)$, azione $Q_1(t)$ e costo.	35
5.6	Confronto PILCO vs isteresi sulla scalinata del Caso 3: temperatura T_1 , azione Q_1 , errore di tracking e costo lossSat.	37

Capitolo 1

Introduzione

1.1 Contesto: l'ascesa del Reinforcement Learning nei controlli automatici

Il *Reinforcement Learning* (RL) ha conosciuto negli ultimi anni una crescita straordinaria, passando da approccio teorico a strumento applicabile in domini complessi quali la robotica, la gestione energetica e il controllo di processo industriale [1]. A differenza del controllo classico, che richiede la conoscenza esplicita del modello del sistema, l'RL consente a un agente di apprendere una politica di controllo ottimale interagendo direttamente con l'ambiente e ricevendo un segnale di rinforzo [2].

In particolare l'applicazione del *Reinforcement Learning* (RL) al controllo di processo ha conosciuto una crescita esponenziale, testimoniata dall'aumento delle pubblicazioni scientifiche sull'argomento [3], [4]. Rispetto al controllo classico, l'RL offre alcuni vantaggi chiave: la capacità di gestire nonlinearità complesse senza richiedere un modello analitico del sistema noto a priori, l'adattabilità a condizioni operative variabili e la possibilità di ottimizzare obiettivi a lungo termine formulati come MDP [5]. Nei sistemi di controllo termico, dove la presenza di nonlinearità, ritardi e disturbi ambientali rende difficile la progettazione di controllori analitici accurati, questi vantaggi risultano particolarmente rilevanti [3].

1.2 Il problema della *sample efficiency*

Uno dei principali ostacoli all'applicazione del RL a sistemi hardware reali è la cosiddetta *sample inefficiency*: gli algoritmi *model-free* — ossia algoritmi che non dispongono di un modello interno della dinamica del sistema — come Q-learning, TD-learning e policy gradient, richie-

dono un numero di interazioni con l'ambiente dell'ordine di migliaia o addirittura milioni di episodi prima di convergere a una politica accettabile [1].

Questo rappresenta un limite critico in scenari fisici, dove ogni interazione ha un costo in termini di tempo, usura dei componenti o rischio di danno. La situazione peggiora ulteriormente in presenza di rumore stocastico non modellato: poiché il rumore è aleatorio, la politica ottima non può essere ricavata semplicemente aumentando il numero di campioni, rendendo necessario un approccio che incorpori esplicitamente l'incertezza nella pianificazione. Un sistema termico come il TCLAB, ad esempio, non può essere sottoposto a migliaia di cicli di riscaldamento e raffreddamento senza subire deterioramento. Gli approcci *model-based*, al contrario, sfruttano un modello interno della dinamica per pianificare traiettorie future, riducendo drasticamente il numero di interazioni necessarie [2].

PILCO [2] si distingue dagli approcci *RL* classici per l'uso di *Gaussian Processes* come modello probabilistico della dinamica, che consente di quantificare esplicitamente l'incertezza epistemica e di propagarla analiticamente durante la pianificazione. Grazie a questa proprietà, PILCO raggiunge prestazioni competitive con un numero di episodi reali notevolmente inferiore rispetto a qualsiasi altro approccio *model-based* [2], rendendolo la scelta naturale per sistemi fisici a bassa dimensionalità dove il costo di ogni interazione è significativo, come nel caso del TCLAB.

1.3 Obiettivo della tesi

Questo lavoro si propone di:

- i. Spiegare brevemente l'algoritmo PILCO, specificandone il suo funzionamento e il modello matematico alla base.
- ii. Spiegare come l'algoritmo PILCO venga implementato in MATLAB, adattandolo alle specificità del sistema TCLAB.
- iii. Valutare le prestazioni di controllo in tre scenari sperimentali di crescente complessità.
- iv. Confrontare l'efficienza campionaria di PILCO rispetto a un approccio *naïf* di riferimento.

1.4 Struttura dell'elaborato

Il resto dell'elaborato è organizzato come segue:

- **Capitolo 2:** introduce i fondamenti teorici: sistemi di controllo classici, *Gaussian Processes* e l'algoritmo PILCO.
- **Capitolo 3:** descrive il sistema sperimentale TCLAB.
- **Capitolo 4:** dettaglia le scelte implementative.
- **Capitolo 5:** riporta i risultati sperimentali nei tre scenari considerati e nel confronto con *isteresi*.
- **Capitolo 6:** discute i risultati, i limiti e un confronto con approcci alternativi e nel confronto con *isteresi*.
- **Capitolo 7:** conclude il lavoro con considerazioni finali e spunti per sviluppi futuri.

Capitolo 2

Stato dell'Arte e Fondamenti Teorici

2.1 Sistemi di controllo e Reinforcement Learning

2.1.1 Controlli classici: isteresi, PID e MPC

Il controllo a *isteresi* (o *bang-bang*), nella sua forma più elementare, rappresenta l'approccio più semplice al controllo termico. Il principio di funzionamento è il seguente: definita una banda di isteresi $[T_{\text{set}} - \delta, T_{\text{set}} + \delta]$ attorno al setpoint, l'attuatore viene commutato tra due stati discreti:

$$u(t) = \begin{cases} Q_{\max} & \text{se } T(t) < T_{\text{set}} - \delta, \\ 0 & \text{se } T(t) > T_{\text{set}} + \delta, \\ u(t^-) & \text{altrimenti,} \end{cases} \quad (2.1)$$

dove δ è la semi-ampiezza della banda e $u(t^-)$ indica il valore precedente del controllo, che rimane invariato all'interno della banda per evitare commutazioni eccessive. Sebbene questo approccio sia estremamente semplice da implementare e intrinsecamente stabile, presenta limitazioni significative che verranno discusse successivamente nel Capitolo 5.

I controllori *Proportional-Integral-Derivative* (PID) rappresentano lo standard industriale per il controllo di processo grazie alla loro semplicità e robustezza. A differenza del controllo a isteresi, il PID produce un segnale di controllo continuo e proporzionale all'errore, eliminando l'oscillazione permanente:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}, \quad e(t) = T_{\text{set}} - T(t). \quad (2.2)$$

Tuttavia, richiedono la taratura manuale dei parametri K_p , K_i , K_d — effettuabile empiricamente, ad esempio con il metodo di Ziegler-Nichols — e non sono ottimali in presenza di forti nonli-

nearità o variazioni significative delle condizioni operative, poiché i parametri sono tipicamente tarati attorno a un singolo punto di lavoro.

Il *Model Predictive Control* (MPC) supera alcune di queste limitazioni integrando un modello della dinamica all'interno di un problema di ottimizzazione a orizzonte finito:

$$\min_{u_0, \dots, u_{T-1}} \sum_{t=0}^T \ell(x_t, u_t) \quad \text{s.t.} \quad x_{t+1} = f(x_t, u_t), \quad u_t \in \mathcal{U}. \quad (2.3)$$

Il limite principale dell'MPC tradizionale è la necessità di disporre di un modello analitico accurato del sistema, la cui identificazione può essere onerosa in presenza di dinamiche complesse o nonlineari.

2.2 Formalizzazione del Problema di Reinforcement Learning

Sebbene questi approcci classici siano consolidati nella pratica industriale, presentano limitazioni comuni: richiedono una conoscenza a priori del sistema (nel caso dell'MPC) oppure non sono in grado di adattarsi a condizioni operative variabili e dinamiche nonlineari complesse (nel caso del PID e dell'isteresi). Un approccio basato sul *Reinforcement Learning* risulta pertanto preferibile in scenari come il TCLAB, dove la dinamica termica è nonlineare, i disturbi ambientali sono imprevedibili e si desidera un controllore che migliori autonomamente le proprie prestazioni attraverso l'esperienza, senza richiedere un modello analitico del sistema noto a priori [3], [6].

2.2.1 Processi Decisionali di Markov

Il problema del controllo ottimo in presenza di incertezza è formalmente inquadrato nel framework dei *Markov Decision Processes* (MDP) [1], [7]. Un MDP è definito dalla tupla $(\mathcal{X}, \mathcal{U}, f, c, \gamma)$, dove:

- $\mathcal{X} \subseteq \mathbb{R}^D$ è lo spazio continuo degli stati;
- $\mathcal{U} \subseteq \mathbb{R}^F$ è lo spazio continuo delle azioni (segnali di controllo);
- $f : \mathcal{X} \times \mathcal{U} \rightarrow \mathcal{X}$ è la funzione di transizione della dinamica del sistema;
- $c : \mathcal{X} \rightarrow [0,1]$ è la funzione di costo immediato;
- $\gamma \in [0,1]$ è il fattore di sconto temporale.

La proprietà di Markov stabilisce che lo stato futuro x_{t+1} dipende esclusivamente dallo stato corrente x_t e dall'azione u_t , e non dalla storia precedente:

$$p(x_{t+1} \mid x_t, u_t, x_{t-1}, u_{t-1}, \dots) = p(x_{t+1} \mid x_t, u_t). \quad (2.4)$$

2.2.2 Politica e Ritorno Atteso

Una *politica* $\pi : \mathcal{X} \rightarrow \mathcal{U}$ è una funzione che mappa ogni stato osservato in un'azione di controllo. L'obiettivo dell'agente RL è trovare la politica ottima π^* che minimizzi il ritorno atteso su un orizzonte finito di T passi:

$$J^\pi(\theta) = \sum_{t=0}^T \mathbb{E}_{x_t} [c(x_t)], \quad x_0 \sim \mathcal{N}(\mu_0, \Sigma_0), \quad (2.5)$$

dove θ sono i parametri della politica e lo stato iniziale x_0 è campionato da una distribuzione gaussiana con media μ_0 e covarianza Σ_0 [2].

2.2.3 Approcci Model-Free e Model-Based

A seconda di come viene affrontata la stima della funzione di transizione f , gli algoritmi RL si dividono in due grandi categorie [1], [6]:

- **Model-free:** la politica ottima viene stimata direttamente dall'interazione con l'ambiente, senza costruire un modello esplicito di f . Algoritmi come Q-learning [8], DDPG [9] e SAC [10] rientrano in questa categoria. Il principale svantaggio è l'elevato numero di campioni richiesti prima della convergenza.
- **Model-based:** viene prima appreso un modello $\hat{f} \approx f$ della dinamica, che viene poi utilizzato per pianificare traiettorie future e ottimizzare la politica senza interagire continuamente con l'ambiente reale [6]. Questo approccio riduce drasticamente il numero di interazioni necessarie, al costo di introdurre un *model bias*: se $\hat{f}$ non è accurato, la politica appresa potrebbe essere subottimale o instabile.

2.2.4 Il Problema del Model Bias

Il *model bias* è il principale limite degli approcci model-based: un modello deterministico $\hat{f}$ che si allontana dalle regioni di addestramento produce predizioni arbitrarie con piena confidenza, portando a politiche che falliscono sul sistema reale [2]. La soluzione proposta da PILCO

consiste nell'usare un modello *probabilistico* $p(x_{t+1} \mid x_t, u_t)$, che invece di restituire una singola predizione fornisce una distribuzione di probabilità sullo stato successivo, quantificando esplicitamente l'incertezza nelle zone inesplorate dello spazio degli stati.

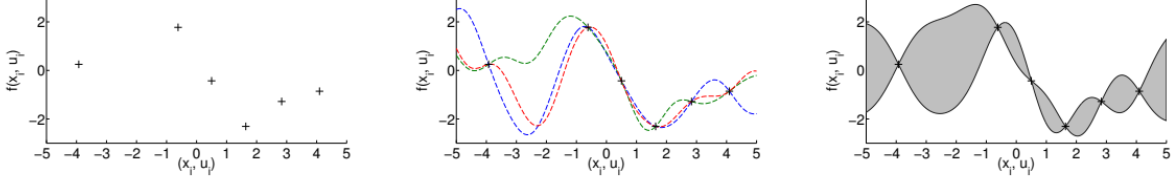


Figura 2.1: Illustrazione del model bias [2]. A sinistra: dataset di transizioni osservate. Al centro: possibili funzioni deterministiche compatibili con i dati — ciascuna produce predizioni arbitrarie fuori dalla zona di training con piena confidenza. A destra: il modello probabilistico GP esprime correttamente l'incertezza nelle zone inesplorate.

In questo modo, l'incertezza del modello viene propagata lungo la traiettoria pianificata e incorporata direttamente nell'ottimizzazione della politica, riducendo il model bias in modo principiato [2], [11].

2.3 Gaussian Processes

2.3.1 Definizione e Intuizione

Un *Gaussian Process* (GP) è un modello probabilistico non parametrico che definisce una distribuzione su funzioni [12]. A differenza dei modelli parametrici classici (come le reti neurali o i modelli lineari), che assumono una forma funzionale fissa e stimano un insieme finito di parametri, un GP non vincola la forma della funzione appresa: ogni possibile funzione ha una certa probabilità di essere quella “vera”, e l'insieme di tutte queste funzioni costituisce la distribuzione a priori.

Intuitivamente, un GP può essere pensato come un'estensione infinito-dimensionale della distribuzione gaussiana multivariata: mentre quest'ultima descrive la distribuzione congiunta di un vettore finito di variabili aleatorie, un GP descrive la distribuzione congiunta di un'intera funzione valutata in un numero arbitrario di punti [12].

Formalmente, un GP è completamente specificato dalla sua funzione media $m(\cdot)$ e dalla sua funzione di covarianza $k(\cdot, \cdot)$:

$$f \sim \mathcal{GP}(m(\cdot), k(\cdot, \cdot)), \quad (2.6)$$

dove $m(\tilde{x}) = \mathbb{E}[f(\tilde{x})]$ rappresenta il valore atteso della funzione in ogni punto (tipicamente si pone $m \equiv 0$ per semplicità), mentre $k(\tilde{x}, \tilde{x}') = \text{cov}[f(\tilde{x}), f(\tilde{x}')]$ misura quanto i valori della funzione in due punti diversi siano correlati tra loro.

2.3.2 Il Kernel e la sua Interpretazione

La funzione di covarianza, o *kernel*, è l'elemento centrale del GP: essa codifica le ipotesi a priori sulla regolarità e la struttura della funzione da apprendere. In PILCO si utilizza il kernel *Squared Exponential* (SE) con *Automatic Relevance Determination* (ARD) [12]:

$$k(\tilde{x}, \tilde{x}') = \alpha^2 \exp\left(-\frac{1}{2}(\tilde{x} - \tilde{x}')^\top \Lambda^{-1}(\tilde{x} - \tilde{x}')\right), \quad (2.7)$$

dove:

- α^2 è la varianza della funzione latente (che sto cercando di apprendere), che controlla l'ampiezza delle fluttuazioni della funzione attorno alla media;
- $\Lambda = \text{diag}(\ell_1^2, \dots, \ell_D^2)$ è una matrice diagonale contenente le *lunghezze caratteristiche* ℓ_i per ciascuna dimensione dell'input.

La lunghezza caratteristica ℓ_i determina quanto velocemente la correlazione tra due punti decresce al crescere della loro distanza nella dimensione i : un valore grande indica che la funzione varia lentamente (è “liscia”) lungo quella dimensione, mentre un valore piccolo indica variazioni rapide. Il meccanismo ARD consente quindi al GP di identificare automaticamente quali dimensioni dell'input sono più rilevanti per predire l'output, assegnando ℓ_i grandi alle dimensioni irrilevanti e ℓ_i piccoli a quelle informative [12].

Il kernel SE garantisce inoltre che la funzione appresa sia infinitamente differenziabile, una proprietà desiderabile per modellare dinamiche fisiche tipicamente continue come quelle termiche del TCLAB.

2.3.3 Predizione con GP

Il punto di forza del GP è la sua capacità di fornire non solo una predizione puntuale, ma una *distribuzione di probabilità completa* sull'output per ogni nuovo input, aggiornando le proprie credenze in modo coerente con i dati osservati tramite il teorema di Bayes.

Dato un insieme di n osservazioni con rumore gaussiano $y_i = f(\tilde{x}_i) + \epsilon_i$, con $\epsilon_i \sim \mathcal{N}(0, \sigma_\epsilon^2)$, raccolte nel dataset $\mathcal{D} = \{(\tilde{x}_i, y_i)\}_{i=1}^n$, la distribuzione predittiva *posteriore* per un nuovo input

$\tilde{x}_*$ è gaussiana [12]:

$$m_f(\tilde{x}_*) = k_*^\top (K + \sigma_\epsilon^2 I)^{-1} \mathbf{y}, \quad (2.8)$$

$$\sigma_f^2(\tilde{x}_*) = k_{**} - k_*^\top (K + \sigma_\epsilon^2 I)^{-1} k_*, \quad (2.9)$$

dove:

- $k_* = [k(\tilde{x}_1, \tilde{x}_*), \dots, k(\tilde{x}_n, \tilde{x}_*)]^\top \in \mathbb{R}^n$ è il vettore delle covarianze tra il nuovo punto $\tilde{x}_*$ e tutti i punti di training;
- $k_{**} = k(\tilde{x}_*, \tilde{x}_*)$ è la varianza a priori nel punto $\tilde{x}_*$;
- $K \in \mathbb{R}^{n \times n}$ è la matrice di Gram con $K_{ij} = k(\tilde{x}_i, \tilde{x}_j)$, che codifica le correlazioni tra tutti i punti di training;
- σ_ϵ^2 è la varianza del rumore di misura, che compare sulla diagonale per regolarizzare il sistema.

La media predittiva $m_f(\tilde{x}_*)$ in (2.8) rappresenta la stima più probabile dell'output e può essere interpretata come una combinazione pesata dei valori osservati, con pesi che dipendono dalla similarità tra $\tilde{x}_*$ e i punti di training misurata dal kernel.

La varianza predittiva $\sigma_f^2(\tilde{x}_*)$ in (2.9) quantifica l'incertezza della predizione: essa è piccola nelle zone densamente campionate (dove il modello è affidabile) e grande nelle zone inesplorate dello spazio degli stati (dove il modello è incerto). Questa proprietà consente a PILCO di distinguere tra *incertezza epistemica* — dovuta alla mancanza di dati, riducibile raccogliendo nuove osservazioni — e *incertezza aleatoria* — dovuta al rumore intrinseco del sistema, irreducibile [12].

Gli iperparametri del GP $\{\alpha^2, \ell_1, \dots, \ell_D, \sigma_\epsilon^2\}$ non vengono fissati a priori, ma ottimizzati massimizzando la *log-verosimiglianza marginale* del modello sui dati di training:

$$\log p(\mathbf{y} \mid \tilde{X}) = -\frac{1}{2} \mathbf{y}^\top (K + \sigma_\epsilon^2 I)^{-1} \mathbf{y} - \frac{1}{2} \log |K + \sigma_\epsilon^2 I| - \frac{n}{2} \log 2\pi, \quad (2.10)$$

bilanciando automaticamente la fedeltà ai dati e la complessità del modello [12].

2.4 L'algoritmo PILCO

PILCO (*Probabilistic Inference for Learning COntrol*) è stato proposto da Deisenroth e Rasmussen nel 2011 [2] come framework di *policy search* model-based che riduce il *model bias* in modo principiato, incorporando esplicitamente l'incertezza del modello GP nella pianificazione

a lungo termine. A differenza degli approcci model-based deterministici, che producono predizioni arbitrarie con piena confidenza nelle zone non osservate, PILCO propaga l'incertezza lungo l'intera traiettoria pianificata, evitando di ottimizzare una politica su predizioni inaffidabili [2].

L'algoritmo alterna due fasi principali: una fase di *apprendimento del modello*, in cui il GP viene aggiornato con tutti i dati raccolti fino a quel momento, e una fase di *ottimizzazione della politica*, in cui la politica viene migliorata sfruttando il modello appreso per simulare traiettorie future senza interagire con il sistema reale.

2.4.1 Modello Probabilistico della Dinamica

Anziché modellare direttamente lo stato successivo x_t , PILCO modella la *differenza di stato* $\delta_t = x_t - x_{t-1}$. Questa scelta è vantaggiosa perché le differenze di stato hanno tipicamente una dinamica più semplice da apprendere rispetto agli stati assoluti, e il GP può sfruttare meglio la struttura locale della dinamica [2].

Per ciascuna dimensione $d = 1, \dots, D$ dello stato, viene addestrato un GP indipendente che prende come input la coppia stato-controllo $\tilde{x}_{t-1} = [x_{t-1}^\top, u_{t-1}^\top]^\top$ e predice la differenza δ_t^d . La distribuzione predittiva dello stato successivo è quindi:

$$p(x_t \mid x_{t-1}, u_{t-1}) = \mathcal{N}(x_t \mid \mu_t, \Sigma_t), \quad \mu_t = x_{t-1} + \mathbb{E}_f[\delta_t], \quad (2.11)$$

dove la media e la varianza sono calcolate tramite le equazioni (2.8)–(2.9) del GP.

I GP per le diverse dimensioni sono addestrati indipendentemente per input deterministici noti, ma diventano correlati quando l'input è incerto — come avviene durante la propagazione della traiettoria — poiché condividono lo stesso input stocastico $\tilde{x}_{t-1}$ [2].

2.4.2 Valutazione Analitica della Politica

Per valutare il ritorno atteso J^π in (2.5), PILCO deve propagare la distribuzione dello stato $p(x_0) \rightarrow p(x_1) \rightarrow \dots \rightarrow p(x_T)$ attraverso il modello GP. È importante chiarire perché questa operazione non è banale, nonostante il GP produca predizioni gaussiane.

Perché il GP produce una gaussiana per input deterministici. Se lo stato corrente $\tilde{x}_{t-1}$ fosse noto esattamente (input deterministico), la distribuzione predittiva del GP sarebbe una gaussiana esatta con media e varianza calcolabili analiticamente tramite le equazioni (2.8)–(2.9). In questo caso, propagare lo stato sarebbe immediato.

Perché la propagazione diventa non gaussiana. Il problema nasce a partire dal secondo passo: lo stato x_{t-1} non è più noto esattamente, ma è esso stesso distribuito come $p(x_{t-1}) =$

$\mathcal{N}(\mu_{t-1}, \Sigma_{t-1})$. Per ottenere la distribuzione dello stato successivo occorre marginalizzare rispetto all'input incerto:

$$p(\delta_t) = \int p(\delta_t | \tilde{x}_{t-1}) p(\tilde{x}_{t-1}) d\tilde{x}_{t-1}. \quad (2.12)$$

Sebbene $p(\delta_t | \tilde{x}_{t-1})$ sia gaussiana per ogni valore fissato di $\tilde{x}_{t-1}$, la sua *media* $m_f(\tilde{x}_{t-1})$ dipende da $\tilde{x}_{t-1}$ in modo **nonlineare** tramite il kernel SE:

$$m_f(\tilde{x}_{t-1}) = k(\tilde{X}, \tilde{x}_{t-1})^\top (K + \sigma_\epsilon^2 I)^{-1} \mathbf{y}, \quad (2.13)$$

dove $k(\tilde{X}, \tilde{x}_{t-1})$ contiene termini esponenziali in $\tilde{x}_{t-1}$. Integrare una funzione esponenziale di una variabile gaussiana produce una distribuzione che **non è gaussiana**: l'integrale in (2.12) è quindi analiticamente intrattabile.

La soluzione di PILCO: moment matching. PILCO risolve questo problema approssimando la distribuzione risultante con una gaussiana tramite *moment matching* esatto: invece di calcolare l'intera distribuzione (intrattabile), si calcolano analiticamente i suoi primi due momenti — media e covarianza — e si approssima il risultato con una gaussiana avente quegli stessi momenti [2]. Questo approccio è più accurato di una linearizzazione locale perché la nonlinearità del GP viene integrata esattamente nel calcolo dei momenti, senza alcuna approssimazione locale.

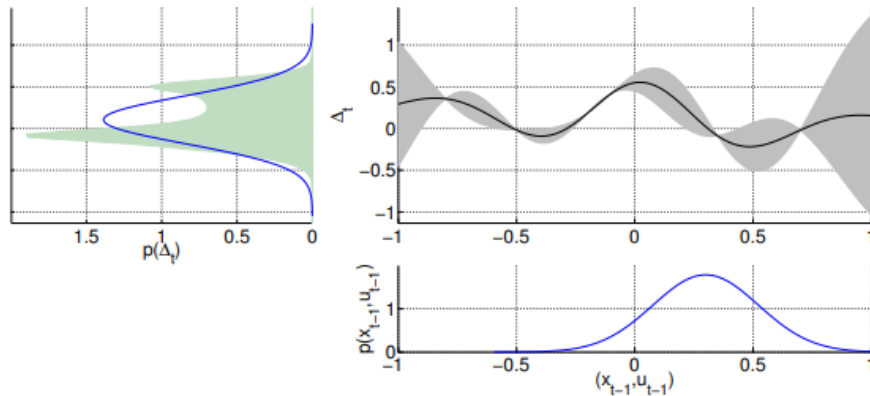


Figura 2.2: Predizione GP con input incerto [2]. La distribuzione di input $p(x_{t-1}, u_{t-1})$ gaussiana (pannello in basso a destra) viene propagata attraverso il modello GP, producendo la distribuzione $p(\delta_t)$ approssimata tramite moment matching (pannello in alto a sinistra).

Le equazioni di propagazione risultanti sono:

$$\mu_t = \mu_{t-1} + \mu_\delta, \quad (2.14)$$

$$\Sigma_t = \Sigma_{t-1} + \Sigma_\delta + \text{cov}[x_{t-1}, \delta_t] + \text{cov}[\delta_t, x_{t-1}], \quad (2.15)$$

dove μ_δ e Σ_δ sono la media e la covarianza della differenza di stato predetta calcolate analiticamente dall'integrale (2.12), e i termini $\text{cov}[x_{t-1}, \delta_t]$ catturano la correlazione tra lo stato corrente e la variazione predetta, necessaria per una propagazione corretta dell'incertezza.

Si noti che Σ_t cresce ad ogni passo temporale: l'incertezza si accumula lungo la traiettoria, riflettendo correttamente il fatto che predizioni a lungo termine sono intrinsecamente meno affidabili di quelle a breve termine. Questo accumulo di incertezza è ciò che penalizza naturalmente le traiettorie che portano il sistema in zone inesplorate, guidando l'esplorazione in modo principiato durante l'ottimizzazione della politica.

2.4.3 Funzione di Costo

Il costo immediato utilizzato in PILCO è una funzione *saturating*:

$$c(x) = 1 - \exp\left(-\frac{\|x - x_{\text{target}}\|^2}{2\sigma_c^2}\right) \in [0,1], \quad (2.16)$$

che penalizza la distanza dallo stato target x_{target} . La scelta di questa funzione non è casuale: a differenza di un costo quadratico $\|x - x_{\text{target}}\|^2$, la funzione saturating è limitata superiormente da 1, rendendola robusta rispetto a stati molto lontani dal target dove il costo quadratico diventerebbe enorme e destabilizzerebbe l'ottimizzazione. Inoltre, il parametro σ_c controlla la larghezza della zona di basso costo attorno al target: un valore piccolo richiede alta precisione, un valore grande è più tollerante.

Il valore atteso del costo rispetto alla distribuzione dello stato $p(x_t) = \mathcal{N}(\mu_t, \Sigma_t)$ può essere calcolato analiticamente, risultando anch'esso in una forma chiusa che dipende solo da μ_t e Σ_t [2].

Si noti che x e x_{target} sono vettori deterministici: $c(x)$ è una funzione scalare definita puntualmente. La distribuzione entra in gioco quando si calcola il valore atteso del costo rispetto alla distribuzione dello stato $p(x_t) = \mathcal{N}(\mu_t, \Sigma_t)$ propagata dal GP:

$$E_t = \mathbb{E}_{x_t}[c(x_t)] = \int c(x_t) \mathcal{N}(x_t | \mu_t, \Sigma_t) dx_t, \quad (2.17)$$

che ammette una forma chiusa grazie alla scelta della funzione saturating, dipendendo solo da μ_t e Σ_t [2].

2.4.4 Miglioramento Analitico della Politica

Il vantaggio cruciale di PILCO rispetto ad altri approcci di *policy search* è la disponibilità del gradiente analitico del ritorno atteso J^π rispetto ai parametri della politica θ . Questo è possibile perché tutte le operazioni — propagazione dei momenti, calcolo del costo atteso — sono differenziabili rispetto a θ .

Applicando la regola della catena ricorsivamente lungo la traiettoria si ottiene:

$$\frac{dJ^\pi}{d\theta} = \sum_{t=0}^T \left(\frac{\partial E_t}{\partial \mu_t} \frac{d\mu_t}{d\theta} + \frac{\partial E_t}{\partial \Sigma_t} \frac{d\Sigma_t}{d\theta} \right), \quad (2.18)$$

dove $E_t = \mathbb{E}_{x_t}[c(x_t)]$ è il costo atteso al passo t . Il gradiente viene propagato all'indietro nel tempo in modo analogo al *backpropagation through time* (BPTT) delle reti neurali ricorrenti, con la differenza che qui il modello della dinamica è un GP probabilistico anziché una rete neurale.

La disponibilità del gradiente analitico consente di utilizzare ottimizzatori di secondo ordine come il *Conjugate Gradient* (CG) o L-BFGS, che convergono molto più rapidamente degli ottimizzatori del primo ordine (come SGD) e sono molto più efficienti rispetto alla stima Monte Carlo del gradiente, la cui varianza cresce rapidamente con il numero di parametri [2].

L'algoritmo completo è riportato in Algorithm 1.

2.4.5 Pseudocodice

Algorithm 1 PILCO — Probabilistic Inference for Learning COntrol

- 1: **Inizializzazione:** campiona $\theta \sim \mathcal{N}(0, I)$; applica segnali di controllo casuali al sistema reale e registra i dati $\mathcal{D} = \{(\tilde{x}_i, y_i)\}$.
 - 2: **repeat**
 - 3: Apprendi il modello GP della dinamica ottimizzando gli iperparametri $\{\alpha^2, \ell_i, \sigma_\epsilon^2\}$ su $\mathcal{D}$ massimizzando la log-verosimiglianza marginale, Eq. (2.10).
 - 4: **repeat**
 - 5: Propaga $p(x_0) \rightarrow p(x_1) \rightarrow \dots \rightarrow p(x_T)$ tramite moment matching analitico, Eq. (2.14)–(2.15).
 - 6: Calcola il ritorno atteso $J^\pi(\theta)$, Eq. (2.5).
 - 7: Calcola il gradiente analitico $dJ^\pi/d\theta$, Eq. (2.18).
 - 8: Aggiorna i parametri θ con CG o L-BFGS.
 - 9: **until** convergenza di θ ; restituisci θ^* .
 - 10: Applica $\pi^* = \pi(\cdot; \theta^*)$ al sistema fisico per un singolo episodio e aggiungi i nuovi dati a $\mathcal{D}$.
 - 11: **until** il task è considerato appreso
-

Osservazioni sull’algoritmo:

- **Riga 1 — Inizializzazione casuale:** i parametri della politica θ vengono inizializzati casualmente e i primi dati vengono raccolti applicando segnali di controllo casuali. Questo garantisce una copertura iniziale dello spazio degli stati prima di qualsiasi ottimizzazione.
- **Riga 3 — Apprendimento del GP:** ad ogni iterazione del loop esterno, il modello GP viene riaddestrato su *tutti* i dati disponibili, inclusi quelli dei rollout precedenti. Gli iperparametri $\{\alpha^2, \ell_i, \sigma_\epsilon^2\}$ vengono ottimizzati automaticamente, senza richiedere taratura manuale.
- **Righe 5–6 — Propagazione e valutazione:** la distribuzione dello stato viene propagata analiticamente lungo l’orizzonte T tramite moment matching. L’incertezza si accumula nel tempo: Σ_t cresce ad ogni passo, riflettendo la minore affidabilità delle predizioni a lungo termine.
- **Riga 7 — Gradiente analitico:** gli approcci Monte Carlo stimano il gradiente campionando N traiettorie reali dal sistema e mediando i risultati [1]. Questo introduce una varianza elevata nella stima — che cresce con il numero di parametri di θ — e richiede un numero proporzionale di interazioni con il sistema fisico. PILCO evita completamente questo problema calcolando il gradiente in forma chiusa tramite la regola della catena applicata ricorsivamente alle equazioni di propagazione dei momenti: il risultato è esatto (a

meno dell'approssimazione gaussiana del moment matching), privo di varianza stocastica e non richiede alcun rollout aggiuntivo sul sistema reale.

- **Riga 10 — Un solo rollout per iterazione:** dopo ogni ottimizzazione della politica, viene eseguito un *unico* episodio reale sul sistema fisico. Questo è il punto chiave della sample efficiency di PILCO: ogni interazione con il sistema reale è preziosa e viene sfruttata al massimo aggiornando il modello GP con i nuovi dati.

Capitolo 3

Il Sistema Sperimentale: TCLab

Il *Temperature Control Lab* (TCLAB) è una piattaforma didattica e di ricerca a basso costo sviluppata da Hedengren et al. all'Università Brigham Young, progettata specificamente per sperimentare algoritmi di controllo in tempo reale su un sistema fisico reale [13]. Grazie alla sua semplicità costruttiva e alle dinamiche termiche ben caratterizzate, il TCLAB rappresenta un banco di prova ideale per validare approcci di controllo basati su RL in condizioni reali, senza i rischi e i costi associati a sistemi industriali complessi.

3.1 Descrizione dell'Hardware

Il TCLAB è realizzato come uno shield per Arduino, ovvero una scheda che si collega direttamente al microcontrollore tramite i pin digitali e analogici, come mostrato in Figura 3.1. Il sistema è composto da:

- **Microcontrollore Arduino Uno:** costituisce il cuore del sistema, gestendo la lettura dei sensori di temperatura e la generazione del segnale PWM (*Pulse Width Modulation*) per il controllo della potenza dei riscaldatori. La comunicazione con il computer ospite avviene tramite interfaccia USB.
- **Due transistor TIP31C BJT** (Q_1, Q_2), che fungono da riscaldatori in package TO-220. I riscaldatori sono alimentati da un'alimentazione da 5 V 2 A per una potenza massima di 10 W. La potenza dissipata è proporzionale al duty cycle del segnale PWM, controllabile nell'intervallo $[0, 100]$ %.
- **Due termistori TMP36GZ** (T_1, T_2), montati in prossimità dei rispettivi transistor tramite epossidico termico. La tensione di uscita del sensore è linearmente proporzionale alla temperatura secondo la relazione $T [^{\circ}\text{C}] = 0.1 \text{ mV} - 50$, senza necessità di calibrazione.

La precisione tipica è $\pm 1^\circ\text{C}$ a temperatura ambiente e $\pm 2^\circ\text{C}$ nell'intervallo operativo $[-40, 150]^\circ\text{C}$.

- **Aletta di raffreddamento passiva** che garantisce il ritorno alla temperatura ambiente quando i riscaldatori sono spenti, rendendo il sistema fisicamente stabile.

I due moduli riscaldatore-sensore sono posizionati in prossimità l'uno dell'altro, in modo da trasferire calore per conduzione, convezione e irraggiamento termico, rendendo il sistema intrinsecamente **MIMO** (*Multiple Input Multiple Output*): due ingressi $[Q_1, Q_2]$ e due uscite $[T_1, T_2]$.

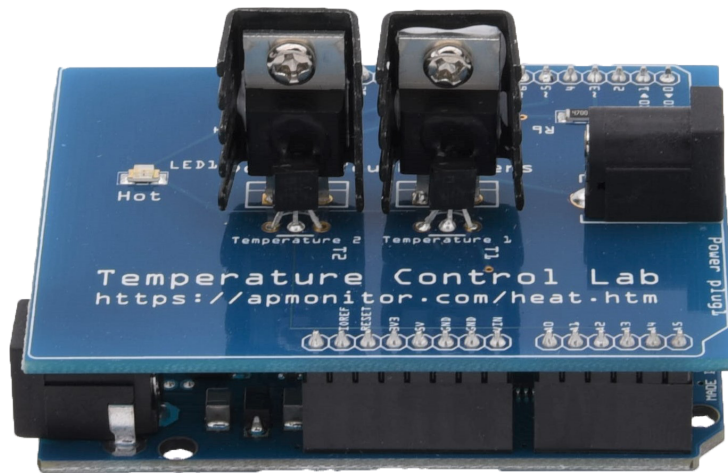


Figura 3.1: Il sistema TCLAB: shield Arduino con i due transistor riscaldatori TIP31C (Q_1, Q_2) e i relativi termistori TMP36GZ (T_1, T_2) [13].

3.2 Modellistica Matematica

Sebbene nell'approccio PILCO il modello della dinamica venga appreso interamente dai dati, è utile conoscere la fisica sottostante per interpretare correttamente i risultati sperimentali [14].

3.2.1 Ipotesi di Modellazione

Il modello fisico del TCLAB si basa sull'ipotesi di *parametri concentrati* (*lumped parameters*): si assume che la distribuzione di temperatura sia sufficientemente uniforme all'interno di ciascun componente da poter essere descritta da un singolo valore scalare. Questa semplificazione è giustificata dalle piccole dimensioni dei transistor e dei termistori [14].

Le ipotesi principali del modello sono:

- I riscaldatori T_{H1}, T_{H2} e i sensori T_{C1}, T_{C2} hanno temperatura uniforme al loro interno.

- I sensori hanno massa e superficie ridotte rispetto ai riscaldatori; la loro dinamica termica è governata principalmente dalla conduzione dall'epossidico termico che li collega ai transistor.
- Gli scambi termici tra i due moduli avvengono per conduzione, convezione e irraggiamento.

3.2.2 Modello Nonlineare Completo

Applicando il bilancio di energia Accumulation = In – Out + Generation – Consumption a ciascun componente, si ottiene il seguente sistema di equazioni differenziali [14]:

$$\begin{cases} \frac{dT_{H1}}{dt} = \frac{1}{mc_p} \left[UA(T_\infty - T_{H1}) + \epsilon\sigma A(T_\infty^4 - T_{H1}^4) + Q_1 \right] \\ \tau_c \frac{dT_{C1}}{dt} = T_{H1} - T_{C1} \end{cases} \quad (3.1)$$

dove:

- mc_p [J K⁻¹]: capacità termica del riscaldatore ($m = 0,004$ kg, $c_p = 500$ J kg⁻¹ K⁻¹);
- UA [W K⁻¹]: coefficiente di perdita convettiva verso l'ambiente ($U = 4.4\text{--}4,6$ W m⁻² K⁻¹, $A = 1 \times 10^{-3}$ m²);
- $\epsilon\sigma A$: termine radiativo secondo la legge di Stefan-Boltzmann ($\epsilon = 0.9$, $\sigma = 5,67 \times 10^{-8}$ W m⁻² K⁻⁴);
- Q_1 [W]: potenza del riscaldatore;
- $\tau_c = 21\text{--}23,3$ s: costante di tempo della conduzione termica attraverso l'epossidico.

La seconda equazione del sistema (3.1) descrive la dinamica del sensore tramite una versione discretizzata della *Legge di Fick* per la conduzione termica: il sensore segue la temperatura del riscaldatore con un ritardo determinato da τ_c .

3.2.3 Semplificazione SISO

Poiché in pratica è disponibile solo la misura del sensore T_{C1} , e come prima approssimazione si può trascurare la dinamica termica del sensore e dell'epossidico, si adotta l'ipotesi [14]:

$$T(t) := T_{C1}(t) \approx T_{H1}(t), \quad (3.2)$$

che semplifica il sistema (3.1) a un'unica equazione nonlineare:

$$\frac{dT(t)}{dt} = \frac{1}{mc_p} \left[UA(T_\infty - T(t)) + \epsilon\sigma A(T_\infty^4 - T^4(t)) + Q_1(t) \right]. \quad (3.3)$$

3.2.4 Linearizzazione e Limiti del Modello Lineare

Linearizzando il modello (3.3) attorno a un punto di equilibrio $(\bar{T}, \bar{Q}_1)$, definendo le variabili di deviazione $\tilde{T} = T - \bar{T}$ e $\tilde{Q}_1 = Q_1 - \bar{Q}_1$, si ottiene:

$$\frac{d\tilde{T}(t)}{dt} = \gamma\tilde{T}(t) + \beta\tilde{Q}_1(t), \quad (3.4)$$

con:

$$\gamma = -\frac{UA}{mc_p} - 4\frac{\epsilon\sigma A\bar{T}^3}{mc_p}, \quad \beta = \frac{1}{mc_p}. \quad (3.5)$$

La funzione di trasferimento corrispondente è:

$$G(s) := \frac{\tilde{T}(s)}{\tilde{Q}_1(s)} = \frac{\beta}{s - \gamma}, \quad (3.6)$$

che è BIBO stabile se e solo se $\gamma < 0$, condizione sempre verificata per i parametri fisici del TCLAB.

Tuttavia, come evidenziato in Figura 3.2, il modello lineare è accurato solo quando Q_1 rimane vicino al punto di linearizzazione $\bar{Q}_1$. Per grandi escursioni di potenza — tipiche di un controllore RL che esplora lo spazio degli stati — l'approssimazione lineare diventa inadeguata, motivando l'uso di un approccio nonlineare come PILCO.

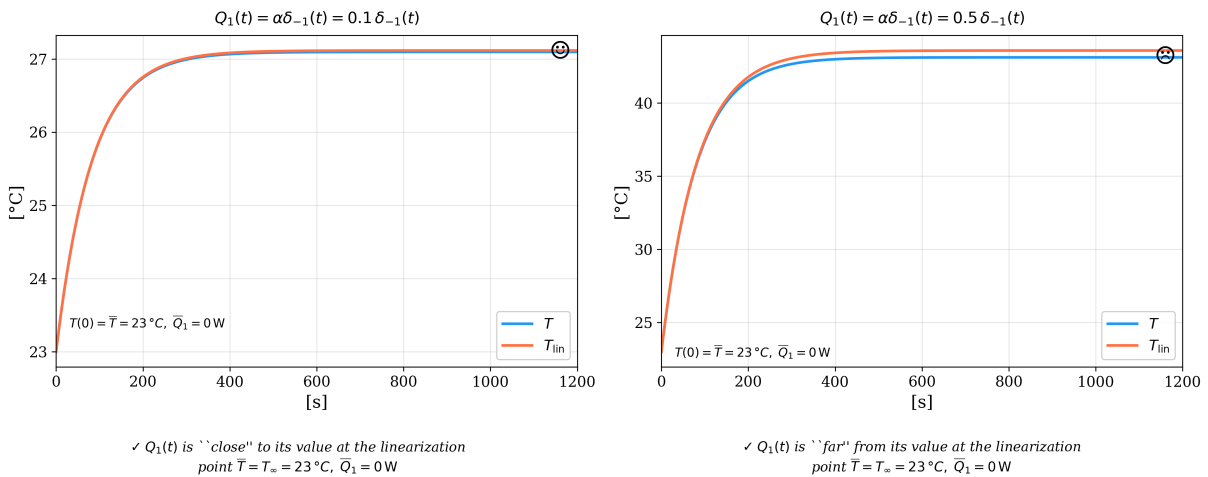


Figura 3.2: Confronto tra modello nonlineare e lineare [14]. Sinistra: Q_1 vicino al punto di linearizzazione — il modello lineare è accurato. Destra: Q_1 lontano dal punto di linearizzazione — il modello lineare sovrastima significativamente la temperatura a regime.

3.3 Perché TCLab per PILCO

Il TCLAB presenta una combinazione di caratteristiche che lo rendono particolarmente adatto a valutare l'efficienza campionaria di PILCO:

- a. **Nonlinearità:** come evidenziato dall'equazione (3.3) e dalla Figura 3.2, il modello lineare è inadeguato per grandi escursioni di temperatura, rendendo necessario un approccio nonlineare appreso dai dati.
- b. **Dinamiche lente:** le costanti di tempo termiche sono dell'ordine di decine di secondi ($\tau_c = 21\text{--}23,3\text{ s}$), consentendo campionamenti a frequenza ridotta ($\Delta t \approx 1\text{ s}$). Un singolo episodio di durata $T = 60\text{ s}$ produce soli 60 campioni, un numero gestibile per il GP la cui complessità scala come $\mathcal{O}(n^3)$.
- c. **Disturbi ambientali:** la temperatura ambiente T_∞ varia nel corso degli esperimenti, introducendo variazioni non modellate che mettono alla prova la robustezza del controllore e la capacità del GP di gestire l'incertezza epistemica.
- d. **Costo delle interazioni:** ogni episodio richiede diversi minuti di tempo reale, rendendo desiderabile minimizzare il numero di rollout — obiettivo primario di PILCO.
- e. **Riproducibilità:** il basso costo e la semplicità d'uso permettono di replicare gli esperimenti in condizioni controllate e di confrontare facilmente PILCO con altri algoritmi di controllo.

Capitolo 4

Implementazione di PILCO sul TCLab

In questo capitolo si descrivono le scelte implementative adottate per applicare l'algoritmo PILCO al controllo termico del TCLAB. L'implementazione è basata sulla repository MATLAB ufficiale di Deisenroth e Rasmussen (github.com/UCL-SML/pilco-matlab) e si articola in tre *casi di studio* con complessità crescente, ciascuno dei quali introduce una nuova sfida senza modificare il nucleo dell'algoritmo.

4.1 Struttura del codice

Il codice è organizzato in moduli riutilizzabili e cartelle specifiche per ogni caso di studio:

- `base/` — loop principale di PILCO: `rollout.m` (esecuzione di un episodio), `trainDynModel.m` (addestramento del GP), `learnPolicy.m` (ottimizzazione della politica), `propagated.m` (propagazione dei momenti con derivate);
- `gp/` — modelli GP con predizione e derivate (`gp1d.m`), addestramento degli iperparametri (`train.m`), GP sparso FITC (`fitc.m`);
- `control/` — politica RBF (`congp.m`) con saturazione (`gSat.m`);
- `loss/` — funzione di costo saturante (`lossSat.m`);
- `pilco_case{1,2,3}/` — file di configurazione, dinamica ODE e script di training/valutazione per ciascun caso.

Ogni caso di studio è definito da tre elementi: un file *settings* (configurazione completa di stato, politica, costo e GP), un file *dynamics* (equazioni ODE del sistema) e uno script di *learn/eval* (loop di training e valutazione).

4.2 Definizione dello stato e dell'azione

Lo spazio degli stati viene progressivamente esteso nei tre casi per gestire scenari di crescente complessità. La Tabella 4.1 riassume le definizioni adottate.

Tabella 4.1: Definizione dello stato per i tre casi di studio.

Caso	Stato	Dim.	Novità
1	$[T_1, T_2]$	2	Baseline
2	$[T_1, T_2, T_{\text{amb}}]$	3	Contesto ambientale
3	$[e, T_2, T_{\text{set}}, Q_2]$	4	Errore + disturbo

Nel Caso 3, lo stato utilizza l'errore di inseguimento $e = T_1 - T_{\text{set}}$ anziché la temperatura assoluta T_1 . Questa scelta rende la funzione di costo *invariante* al setpoint: il target diventa $[0,0,0,0]^\top$ indipendentemente dal valore corrente di T_{set} , consentendo a una singola politica di gestire setpoint diversi senza modificare la struttura del costo.

Lo spazio delle azioni corrisponde alla potenza del riscaldatore 1:

$$u_t = Q_1(t) \in [0,100] \text{ [\%]}. \quad (4.1)$$

Internamente, PILCO rappresenta l'azione nell'intervallo simmetrico $[-50, +50]$ tramite la funzione di saturazione gSat , e la dinamica ODE ricostruisce la potenza fisica come $Q_1 = u + 50$.

4.3 Funzione di costo

Si adotta la funzione di costo *saturating* descritta in (2.16):

$$c(x_t) = 1 - \exp\left(-\frac{1}{2} (x_t - z)^\top W (x_t - z)\right) \in [0,1], \quad (4.2)$$

dove z è lo stato target e W è una matrice diagonale di pesi che seleziona le componenti rilevanti dello stato. In tutti e tre i casi, il costo penalizza unicamente la variabile controllata (T_1 nei Casi 1–2, l'errore e nel Caso 3), assegnando peso zero alle restanti componenti. La Tabella 4.2 riassume la configurazione del costo per ciascun caso.

Tabella 4.2: Configurazione della funzione di costo nei tre casi.

Caso	Target z	Pesi W
1	$[50, 50]^\top$	$\text{diag}(1, 0)$
2	$[50, 0, 0]^\top$	$\text{diag}(1, 0, 0)$
3	$[0, 0, 0, 0]^\top$	$\text{diag}(1, 0, 0, 0)$

4.4 Architettura della politica

In tutti e tre i casi si utilizza un controllore nonlineare basato su una rete a funzioni di base radiali (RBF), implementato come GP sulla politica (`cong.p.m`):

$$\pi(x_t; \theta) = \sum_{i=1}^{n_c} w_i k(x_t, c_i), \quad k(x, c) = \sigma^2 \exp\left(-\frac{1}{2}(x - c)^\top \Lambda^{-1}(x - c)\right), \quad (4.3)$$

dove c_i sono i centri delle basi, w_i i pesi, Λ la matrice delle lunghezze caratteristiche e σ^2 la varianza del segnale. L'output viene saturato nell'intervallo $[-50, +50]$ tramite `gSat`, garantendo che la potenza risultante sia sempre in $[0, 100]\%$.

Un aspetto chiave è la scelta degli *input alla politica* (`pol.i`), che determina quali variabili di stato la politica osserva per decidere l'azione:

- **Caso 1:** $[T_1, T_2]$ — la politica vede entrambe le temperature;
- **Caso 2:** $[T_1, T_{\text{amb}}]$ — T_2 viene esclusa dalla politica (non serve per decidere Q_1) ma resta nel GP della dinamica. Questa riduzione da $\mathbb{R}^3$ a $\mathbb{R}^2$ accelera la convergenza;
- **Caso 3:** $[e, T_{\text{set}}]$ — la politica vede l'errore e il setpoint corrente, consentendo di adattare la potenza alle perdite termiche che dipendono nonlinearmemente dal livello di temperatura.

I centri RBF ($n_c = 20\text{--}25$) vengono inizializzati campionando da una distribuzione gaussiana centrata nello spazio operativo atteso, con varianza sufficiente a coprire il range delle variabili in gioco.

4.5 Modello GP della dinamica

Per ciascuna dimensione dello stato, un GP indipendente apprende la *differenza di stato* $\Delta x_t = x_t - x_{t-1}$ a partire dagli input $\tilde{x}_{t-1} = [x_{t-1}^\top, u_{t-1}^\top]^\top$. Questa scelta semplifica l'apprendimento: le differenze hanno tipicamente una dinamica più regolare rispetto ai valori assoluti.

Il kernel utilizzato è il Squared Exponential con ARD (descritto nella Sezione 2.3), e gli iperparametri vengono ottimizzati ad ogni iterazione di PILCO massimizzando la log-verosimiglianza marginale (2.10).

Quando il dataset supera qualche centinaio di transizioni, viene attivato automaticamente il GP sparso (FITC), che riduce la complessità da $\mathcal{O}(n^3)$ a $\mathcal{O}(nm^2)$ con m punti induttori ($m = 200\text{--}300$), mantenendo un'approssimazione accurata della distribuzione predittiva.

La Tabella 4.3 mostra come la dimensionalità degli input al GP cresca con la complessità del caso.

Tabella 4.3: Dimensionalità degli input al GP della dinamica.

Caso	Input GP	Dim.
1	$[T_1, T_2, u]$	3
2	$[T_1, T_2, T_{\text{amb}}, u]$	4
3	$[e, T_2, T_{\text{set}}, Q_2, u]$	5

4.6 Dinamica ODE del sistema

La simulazione del sistema avviene tramite l'integrazione ODE (ode45) del modello fisico del TCLAB descritto nel Capitolo 3. Ogni caso di studio definisce una propria funzione ODE che implementa il bilancio energetico:

$$\frac{dT_i}{dt} = \frac{1}{mc_p} [UA(T_a - T_i) + \epsilon\sigma A(T_a^4 - T_i^4) + \text{scambio}_{12} + \alpha_i Q_i], \quad (4.4)$$

con le seguenti varianti tra i casi:

- **Caso 1:** $T_{\text{amb}} = 25^\circ\text{C}$ fisso, $Q_2 = 0$;
- **Caso 2:** T_{amb} letto dal terzo elemento dello stato (z_3), con $dT_{\text{amb}}/dt = 0$ (costante nell'episodio ma variabile tra episodi);
- **Caso 3:** stato $[e, T_2, T_{\text{set}}, Q_2]$, dove T_1 viene ricostruito come $T_1 = e + T_{\text{set}}$, e Q_2 agisce come disturbo sull'heater 2. Le derivate di T_{set} e Q_2 sono nulle (costanti nell'episodio).

4.7 Loop di addestramento

Il loop di addestramento segue la struttura dell'Algoritmo 1 e si compone di tre fasi:

Fase 1 — Rollout casuali. Vengono eseguiti J episodi con azioni casuali per raccogliere un dataset iniziale. Nel Caso 1, $J = 15$ rollout con condizioni fisse. Nei Casi 2 e 3, i rollout vengono ciclati sulle diverse condizioni operative (T_{amb} o T_{set}) per garantire una copertura uniforme dello spazio degli stati fin dall'inizio.

Fase 2 — Iterazioni PILCO. Per $N = 5$ iterazioni si eseguono in sequenza:

1. `trainDynModel`: addestramento del GP su tutti i dati raccolti fino a quel momento;
2. `learnPolicy`: ottimizzazione della politica tramite backpropagation attraverso il GP, utilizzando L-BFGS con un massimo di 150 *line search*;
3. Rollout con la politica ottimizzata: nel Caso 1 un singolo episodio, nei Casi 2–3 un episodio per ciascuna condizione operativa di training.

La scelta di ciclare su *tutte* le condizioni operative ad ogni iterazione PILCO (anziché addestrare su una condizione alla volta) è fondamentale per evitare il *catastrophic forgetting*: se la politica fosse ottimizzata prima solo per $T_{\text{amb}} = 25^\circ\text{C}$ e poi solo per $T_{\text{amb}} = 40^\circ\text{C}$, l’ottimizzatore sovrascriverebbe i pesi della RBF utili per la prima condizione, perdendo la capacità di controllarla.

Fase 3 — Salvataggio. Al termine del training, la politica addestrata viene salvata in un file `.mat` contenente tutti i parametri necessari per la valutazione.

4.8 Parametri dell’esperimento

La Tabella 4.4 riassume i principali parametri dell’implementazione, comuni a tutti e tre i casi salvo dove indicato.

Tabella 4.4: Parametri dell’implementazione di PILCO su TCLAB.

Parametro	Valore	Descrizione
Δt	20 s	Passo di campionamento
T_{ep}	600 s	Durata episodio
H	30	Step per episodio
n_c	20–25	Centri RBF della politica
Ottimizzatore	L-BFGS	Aggiornamento della politica
J	12–15	Rollout casuali iniziali
N	5	Iterazioni PILCO
maxU	50	Saturazione azione: $[-50, +50]$

Capitolo 5

Risultati Sperimentali e Confronto con Isteresi

In questo capitolo si presentano i risultati ottenuti nei tre scenari di crescente complessità, seguiti dal confronto diretto tra PILCO e un controllore a isteresi.

5.1 Caso 1 — Regolazione a Setpoint Fisso

5.1.1 Setup sperimentale

Nel primo scenario si opera in condizioni nominali: setpoint costante $T_{\text{set}} = 50^\circ\text{C}$, temperatura ambiente stabile $T_{\text{amb}} = 25^\circ\text{C}$ e nessun disturbo esterno ($Q_2 = 0$). Lo stato è bidimensionale $[T_1, T_2]$ e la politica mappa $[T_1, T_2] \rightarrow Q_1$. L'obiettivo è verificare che PILCO converga a un controllore stabile con un numero minimo di interazioni.

Il training consiste in $J = 15$ rollout casuali seguiti da $N = 5$ iterazioni PILCO, per un totale di 20 episodi da 600 s ciascuno — corrispondenti a circa 200 min di interazione complessiva con il sistema.

5.1.2 Risultati

La Figura 5.1 mostra l'evoluzione del training. Durante i rollout casuali, la temperatura oscilla senza direzione e il costo cumulativo rimane elevato. Già dalla prima iterazione PILCO, la politica impara a riscaldare il sistema verso il setpoint. Dopo $N = 5$ iterazioni, il controllore raggiunge $T_1 \approx T_{\text{set}}$ con un errore a regime stazionario contenuto entro $\pm 1^\circ\text{C}$.

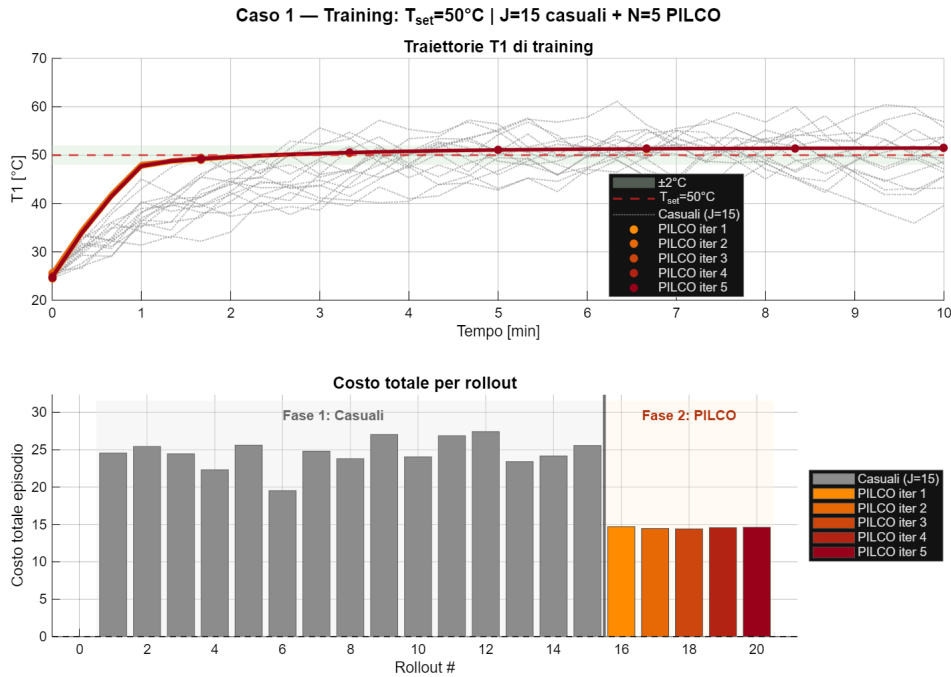


Figura 5.1: Caso 1 — Evoluzione del training: traiettorie $T_1(t)$ e costo cumulativo per i rollout casuali (grigio) e le iterazioni PILCO (colore).

Il segnale di controllo $Q_1(t)$, mostrato in Figura 5.2, presenta un comportamento fisicamente coerente: potenza elevata nelle fasi iniziali di riscaldamento, seguita da una modulazione più fine in prossimità del setpoint, dove le perdite per convezione e irraggiamento bilanciano il calore generato.

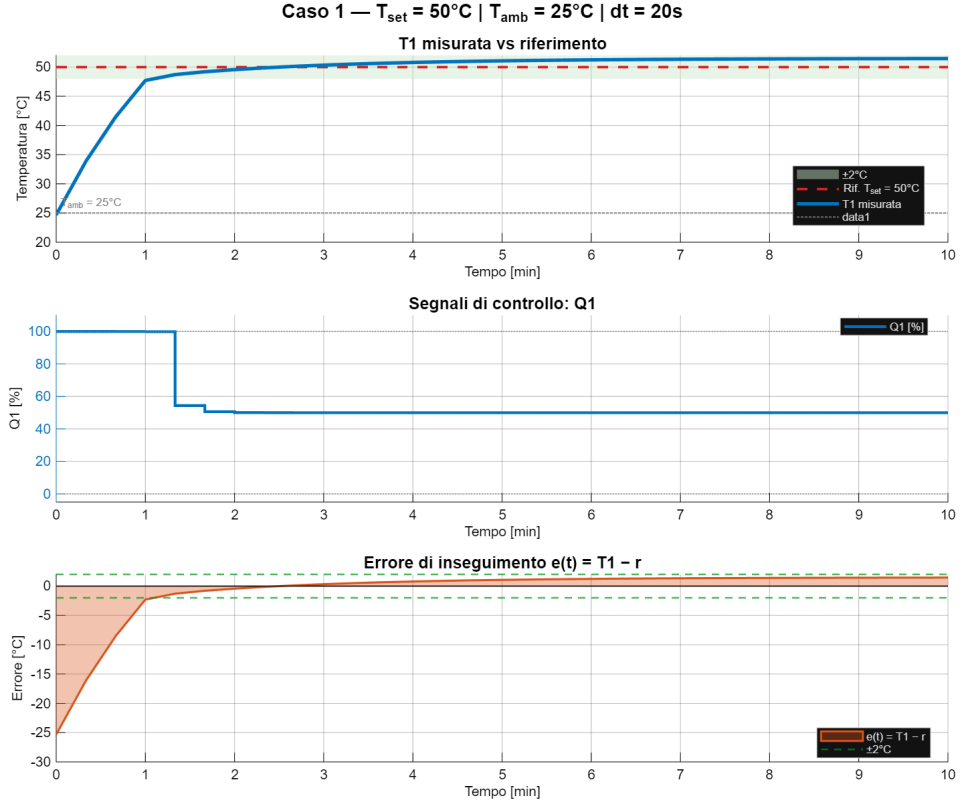


Figura 5.2: Caso 1 — Ultimo rollout PILCO: profilo $T_1(t)$, azione $Q_1(t)$, costo per step e errore di inseguimento.

Il costo lossSat decresce progressivamente lungo le iterazioni, confermando la convergenza dell'algoritmo. La varianza del GP è inizialmente elevata nelle zone inesplorate e si riduce con l'accumulo dei dati, dimostrando la capacità del modello di distinguere le aree a incertezza epistemica elevata da quelle ben coperte.

5.2 Caso 2 — Robustezza alla Temperatura Ambiente

5.2.1 Setup sperimentale

Il setpoint rimane costante ($T_{\text{set}} = 50^\circ\text{C}$), ma la temperatura ambiente viene variata tra episodi di training: $T_{\text{amb}} \in \{25, 35, 40, 30\}^\circ\text{C}$. Lo stato è esteso a tre dimensioni $[T_1, T_2, T_{\text{amb}}]$, dove T_{amb} è trattato come stato con $dT_{\text{amb}}/dt = 0$ (costante nell'episodio). La politica mappa $[T_1, T_{\text{amb}}] \rightarrow Q_1$, escludendo T_2 dallo spazio degli input per ridurre la dimensionalità.

Lo stato iniziale di ogni episodio riflette l'equilibrio termico con l'ambiente: $T_1 = T_2 = T_{\text{amb}}$. Questa scelta è fisicamente realistica — un sistema spento in un ambiente a 35°C parte

con tutte le temperature a 35°C — e arricchisce il dataset del GP con traiettorie di riscaldamento diverse.

Il training utilizza $J = 12$ rollout casuali ciclati sulle quattro temperature ambiente, seguiti da $N = 5$ iterazioni PILCO con 4 rollout ciascuna (uno per ogni T_{amb}), per un totale di 32 episodi.

5.2.2 Risultati

La politica apprende a modulare Q_1 in funzione di T_{amb} : a $T_{\text{amb}} = 25^\circ\text{C}$ (condizione fredda) applica potenze più elevate rispetto a $T_{\text{amb}} = 40^\circ\text{C}$ (condizione calda), dove le perdite termiche verso l'ambiente sono ridotte. Questo comportamento adattivo è reso possibile dalla struttura della rete RBF, i cui neuroni attivano risposte differenti a seconda della posizione nello spazio $[T_1, T_{\text{amb}}]$ (Figura 5.3).

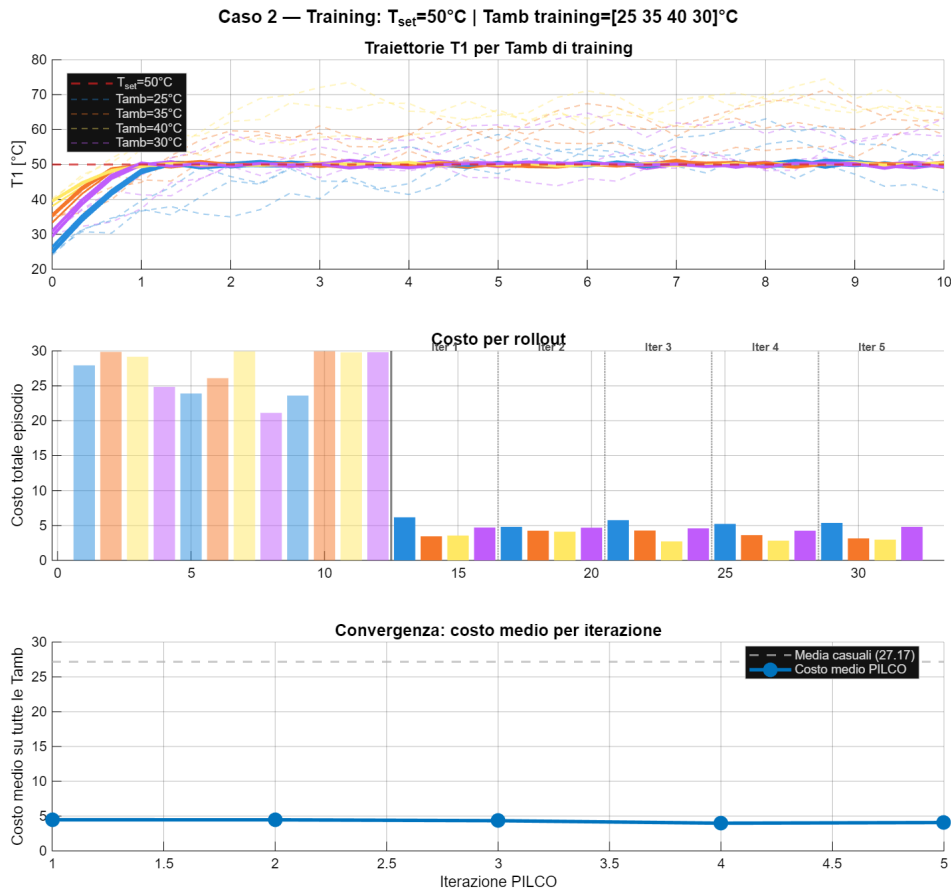


Figura 5.3: Caso 2 — Evoluzione del training sulle quattro temperature ambiente.

L'incertezza del GP aumenta in corrispondenza di condizioni operative non osservate durante il training, fornendo un indicatore naturale dell'affidabilità delle predizioni. Nonostante ciò, il controllore mantiene la temperatura entro un intorno ragionevole dal setpoint anche in condizioni di interpolazione (T_{amb} compreso tra i valori di training).

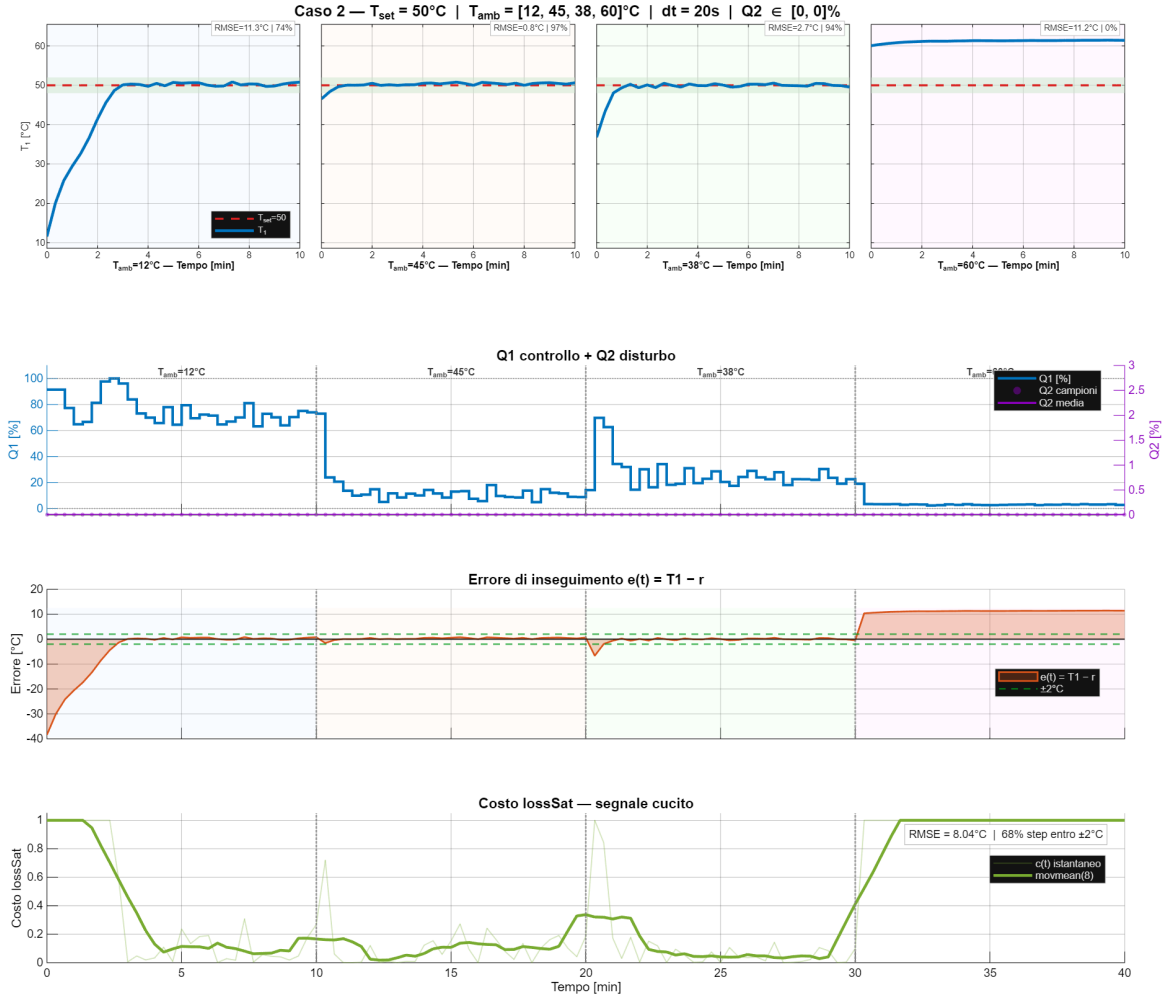


Figura 5.4: Caso 2 — Valutazione: pannello superiore con $T_1(t)$ per ogni T_{amb} di test, segnale di controllo Q_1 e disturbo Q_2 , errore di tracking e costo lossSat sull'intero episodio cucito.

La Figura 5.4 mostra la valutazione finale della politica su una sequenza di segmenti con temperature ambiente diverse. La riga superiore riporta la traiettoria di T_1 per ciascun valore di T_{amb} : in ogni pannello la temperatura converge verso il setpoint di 50°C , con metriche RMSE e percentuale di permanenza entro $\pm 2^{\circ}\text{C}$ calcolate individualmente. La riga centrale mostra come il segnale di controllo Q_1 si adatti automaticamente: nelle condizioni più fredde ($T_{\text{amb}} = 25^{\circ}\text{C}$) la potenza di regime è più elevata, mentre a $T_{\text{amb}} = 40^{\circ}\text{C}$ la politica modula Q_1 a valori inferiori, coerentemente con la riduzione delle perdite termiche verso l'ambiente. L'errore di tracking,

mostrato nella riga inferiore, resta prevalentemente contenuto entro la banda $\pm 2^\circ\text{C}$, con picchi limitati ai transitori iniziali di ciascun segmento.

Le prestazioni in condizioni di *extrapolazione* — ovvero per valori di T_{amb} significativamente al di fuori del range di training — sono inevitabilmente più limitate, poiché il GP non dispone di dati per vincolare le proprie predizioni in quelle regioni.

5.3 Caso 3 — Inseguimento del Riferimento Variabile

5.3.1 Setup sperimentale

In questo scenario il setpoint T_{set} varia tra gli episodi di training: $T_{\text{set}} \in \{35, 60, 24, 43, 80\}^\circ\text{C}$. Lo stato è esteso a quattro dimensioni $[e, T_2, T_{\text{set}}, Q_2]$, dove $e = T_1 - T_{\text{set}}$ è l'errore di inseguimento e Q_2 è un disturbo costante per episodio sull'heater 2.

L'inclusione di setpoint inferiori alla temperatura iniziale ($T_{\text{set}} = 24^\circ\text{C} < T_1^{\text{init}} = 25^\circ\text{C}$) è una scelta deliberata: genera episodi con $e > 0$ (il sistema è più caldo del target), permettendo al GP di apprendere la dinamica di raffreddamento passivo. Senza questi dati, la politica sarebbe impreparata a gestire le discese di setpoint durante la valutazione.

La valutazione avviene su una *scalinata* di gradini mai visti durante il training: la sequenza tipica prevede salite e discese di setpoint, con lo stato fisico che viene propagato continuamente da un gradino al successivo.

5.3.2 Risultati

La Figura 5.5 mostra i risultati della valutazione sulla scalinata. La politica riesce a inseguire i gradini di setpoint con un tempo di assestamento che dipende dall'entità della variazione richiesta. Nelle fasi di *riscaldamento* (gradini in salita), la politica applica Q_1 elevato per accelerare la transizione e lo riduce man mano che T_1 si avvicina al riferimento. Nelle fasi di *raffreddamento* (gradini in discesa), la politica imposta $Q_1 \approx 0$ e il sistema si raffredda passivamente per convezione e irraggiamento — un comportamento appreso grazie agli episodi con $T_{\text{set}} < T_1^{\text{init}}$ durante il training.

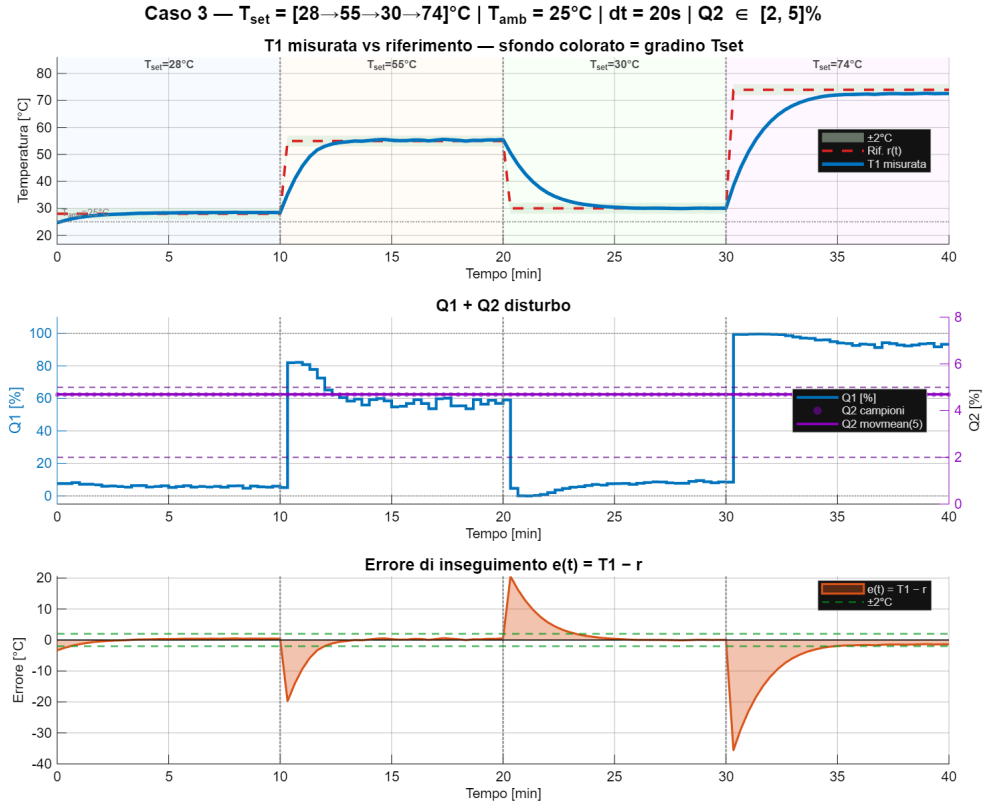


Figura 5.5: Caso 3 — Scalinata di valutazione: profilo $T_1(t)$ vs riferimento, errore $e(t)$, azione $Q_1(t)$ e costo.

Il disturbo Q_2 introduce un riscaldamento parassita sull'heater 2 che, attraverso lo scambio termico tra i due moduli, influenza indirettamente T_1 . Il GP della dinamica modella questo effetto come parte della sua rappresentazione della fisica, senza che la politica debba reagirvi esplicitamente (ricordiamo che Q_2 non è tra gli input della politica, $\text{poli} = [1, 3]$).

I setpoint al di fuori del range di training (ad esempio $T_{\text{set}} > 60^\circ\text{C}$) rappresentano la prova più impegnativa: l'errore di tracking aumenta e il tempo di assestamento si allunga, poiché il GP extrapola in regioni dello spazio degli stati non coperte dai dati di addestramento.

5.4 Confronto PILCO vs Controllore a Isteresi

Per fornire un termine di paragone quantitativo, PILCO viene confrontato con un controllore a isteresi — l’approccio più semplice al controllo termico, descritto nel Capitolo 2.

5.4.1 Configurazione del confronto

Il confronto avviene sullo scenario del Caso 3 (scalinata di setpoint variabile con disturbo Q_2), utilizzando condizioni identiche per entrambi i controllori:

- Stessa sequenza di gradini: $[25, 45, 35, 55]$ °C con durata di 400 s per gradino;
- Stesso disturbo Q_2 (generato con $\text{rng}(42)$ per riproducibilità);
- Stessa dinamica ODE e rumore di misura;
- Stessa funzione di costo lossSat per il calcolo delle metriche.

Il controllore a isteresi opera con una logica on/off:

$$Q_1(t) = \begin{cases} 100 \% & \text{se } T_1(t) < T_{\text{set}} - \delta, \\ 0 \% & \text{se } T_1(t) > T_{\text{set}} + \delta, \\ Q_1(t^-) & \text{altrimenti,} \end{cases} \quad (5.1)$$

con banda di isteresi $2\delta = 4$ °C ($\delta = 2$ °C).

5.4.2 Risultati del confronto

La Figura 5.6 mette a confronto le prestazioni dei due controllori sulla stessa scalinata di setpoint. La Tabella 5.1 riassume le metriche principali.

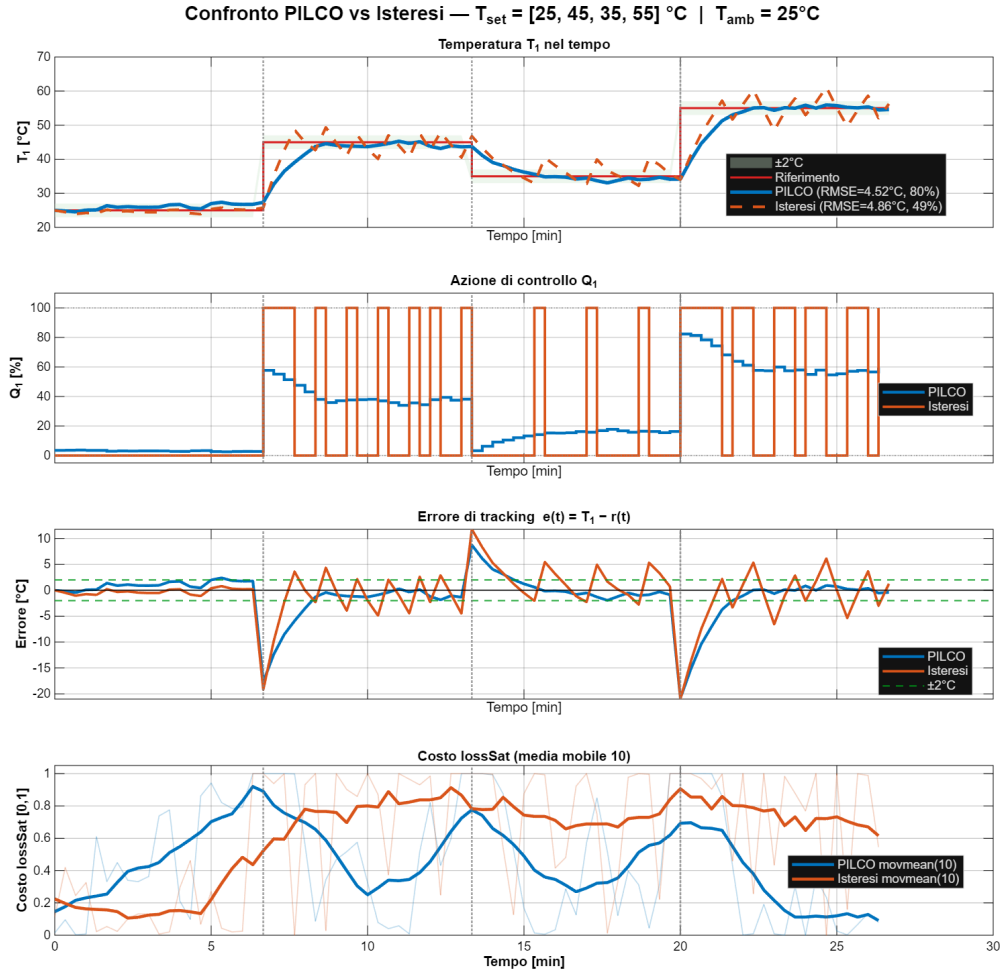


Figura 5.6: Confronto PILCO vs isteresi sulla scalinata del Caso 3: temperatura T_1 , azione Q_1 , errore di tracking e costo lossSat.

Tabella 5.1: Metriche di confronto PILCO vs isteresi.

Metrica	PILCO	Isteresi
RMSE tracking $[^\circ\text{C}]$	4,52	4,86
$\max e(t) [^\circ\text{C}]$	20,84	21,01
% entro $\pm 2^\circ\text{C}$	80,0 %	48,8 %
Costo medio lossSat	0,4481	0,6224

Dal confronto emergono differenze significative nel comportamento dei due controllori:

Segnale di controllo. L'isteresi produce un segnale Q_1 che commuta tra 0 % e 100 %, generando oscillazioni permanenti della temperatura attorno al setpoint con ampiezza pari alla banda di isteresi ($\pm 2^\circ\text{C}$). Queste commutazioni rapide e di grande ampiezza possono ridurre la vita utile degli attuatori in un contesto industriale.

Al contrario, PILCO produce un segnale di controllo *modulato*: la potenza viene regolata in modo continuo e progressivo, senza commutazioni brusche. Questo comportamento è intrinsecamente più favorevole sia per la precisione del controllo sia per la protezione degli attuatori.

Precisione a regime. L'isteresi, per sua natura, non può eliminare l'oscillazione permanente: la temperatura resta confinata nella banda $[T_{\text{set}} - \delta, T_{\text{set}} + \delta]$, senza mai convergere esattamente al setpoint. PILCO, disponendo di un segnale di controllo continuo, può in linea di principio raggiungere un errore a regime arbitrariamente piccolo, limitato solo dalla precisione del modello GP e dal rumore di misura.

Risposta ai transitori. Durante i cambi di setpoint (gradini della scalinata), l'isteresi reagisce immediatamente applicando $Q_1 = 100\%$ (riscaldamento) o $Q_1 = 0\%$ (raffreddamento) con la massima potenza disponibile. PILCO adotta una strategia più graduale, modulando la potenza in funzione della distanza dal target e della dinamica predetta dal GP. Questo può risultare leggermente più lento nella fase iniziale del transitorio, ma evita overshoot e oscillazioni successive.

Robustezza al disturbo Q_2 . Nessuno dei due controllori “vede” direttamente Q_2 : l'isteresi lo ignora completamente (agisce solo su T_1), mentre PILCO ha implicitamente catturato il suo effetto nel modello GP della dinamica. Questo conferisce a PILCO una maggiore capacità di compensare l'effetto di Q_2 sulla temperatura T_1 , anche se in modo indiretto.

Semplicità vs intelligenza. L'isteresi è implementabile in poche righe di codice, non richiede alcun addestramento e funziona fin dal primo istante. PILCO richiede un investimento iniziale significativo (rollout casuali + iterazioni di ottimizzazione) prima di diventare operativo, ma ripaga questo investimento con prestazioni superiori una volta addestrato.

Capitolo 6

Discussione dei Risultati

6.1 Efficienza di campionamento

L'aspetto più rilevante dei risultati ottenuti riguarda il numero di interazioni con il sistema necessarie per ottenere un controllore funzionante. In tutti e tre i casi, PILCO converge a una politica efficace con un totale di 20–40 episodi (rollout casuali inclusi), corrispondenti a circa 200–400 minuti di interazione complessiva.

Questo risultato è coerente con quanto riportato nella letteratura originale [2], dove PILCO supera di almeno un ordine di grandezza i metodi *model-free* come nel task *cart-pole*. Nel contesto del TCLAB, dove ogni episodio richiede 10 min di tempo reale, la differenza è ancora più significativa: un algoritmo *model-free* come Q-learning richiederebbe centinaia o migliaia di episodi, rendendo l'esperimento impraticabile su hardware fisico.

La chiave di questa efficienza risiede nel doppio ruolo del GP: esso funge simultaneamente da modello della dinamica (per la simulazione interna durante l'ottimizzazione della politica) e da “segnale di allerta” sulle regioni dello spazio degli stati dove le predizioni non sono affidabili (varianza predittiva elevata).

6.2 Interpretazione dell'incertezza

La varianza del GP ha svolto un ruolo fondamentale nel guidare l'apprendimento: nelle prime iterazioni, il modello assegna alta incertezza alle zone dello spazio degli stati non ancora visitate, incentivando la politica a esplorare traiettorie più informative tramite la pianificazione probabilistica.

Nei Casi 2 e 3, l'aumento della varianza in corrispondenza di condizioni operative non viste durante il training (ad esempio T_{amb} o T_{set} al di fuori del range di addestramento) è un segnale naturale per identificare quando il modello non è affidabile e occorre raccogliere nuovi dati.

Questa proprietà è intrinseca al framework GP e non richiede alcun meccanismo di esplorazione esplicita.

6.3 Analisi del confronto PILCO vs isteresi

Il confronto con il controllore a isteresi evidenzia il diverso compromesso tra semplicità implementativa e qualità del controllo. Prima di analizzare i singoli aspetti, è utile evidenziare la differenza concettuale fondamentale tra i due approcci.

6.3.1 Decisione locale vs decisione globale

Il controllore a isteresi opera in modo puramente *reattivo e locale nel tempo*: ad ogni istante osserva unicamente l'errore corrente $e(t) = T_{\text{set}} - T_1(t)$ e produce una decisione binaria — riscaldatore acceso ($Q_1 = 100\%$) o spento ($Q_1 = 0\%$) — senza alcuna memoria del passato né previsione del futuro. La logica è completamente contenuta nella regola a soglia:

$$Q_1(t) = \begin{cases} 100\% & \text{se } e(t) > +\delta, \\ 0\% & \text{se } e(t) < -\delta. \end{cases} \quad (6.1)$$

Questa semplicità è al tempo stesso il punto di forza e il limite dell'isteresi: non è richiesta alcuna conoscenza del sistema, ma il controllore non può anticipare l'evoluzione della temperatura né distinguere tra condizioni operative diverse che producono lo stesso errore istantaneo.

PILCO, al contrario, prende decisioni *globali*: la politica $\pi(x_t; \theta)$ non è una regola a soglia sull'errore corrente, ma una funzione appresa durante il training che incorpora implicitamente la comprensione della dinamica del sistema. Quando PILCO osserva un errore elevato in una certa “direzione” dello spazio degli stati (ad esempio T_1 molto inferiore al setpoint con T_{amb} bassa), la rete RBF attiva neuroni specifici per quella regione e produce un'azione calibrata — non semplicemente “acceso” — tenendo conto delle conseguenze a lungo termine dell'azione scelta, così come le ha apprese dal modello GP della dinamica.

6.3.2 Apprendimento senza conoscenza a priori

Un aspetto notevole è che PILCO raggiunge queste prestazioni *senza alcuna informazione a priori sul sistema*. La politica non è progettata con la conoscenza del modello termico, dei parametri fisici o della struttura delle equazioni differenziali: tutto viene appreso esclusivamente dai dati raccolti durante i rollout. Il GP funge da “simulatore appreso”, catturando le relazioni di causa-effetto tra azioni e stati successivi in un contesto dinamico.

Questa capacità di inferire la dinamica del sistema da poche decine di episodi è il risultato principale del framework model-based: il GP non si limita a memorizzare le transizioni osservate, ma costruisce un modello predittivo interpolante che generalizza a stati e azioni mai visti, quantificando al contempo la propria incertezza nelle zone inesplorate.

6.3.3 Considerazioni sulla comparabilità

Si potrebbe obiettare che il confronto tra PILCO — un algoritmo con numerosi gradi di libertà (centri RBF, pesi, iperparametri del GP) — e un controllore a isteresi con un solo parametro (δ) sia intrinsecamente sbilanciato. Questa osservazione è corretta: i due controllori si collocano a estremi opposti dello spettro di complessità e il confronto non intende suggerire che l'isteresi sia un termine di paragone adeguato per valutare l'ottimalità di PILCO.

L'obiettivo del confronto è piuttosto *pedagogico e illustrativo*: mostrare concretamente quali vantaggi si ottengono passando da un approccio reattivo senza modello a un approccio predittivo con modello appreso, evidenziando il salto qualitativo nella modulazione del controllo, nella capacità di adattamento e nella gestione di scenari complessi. Il fatto che PILCO riesca a catturare relazioni di causa-effetto complesse — come la dipendenza nonlineare delle perdite radiative dalla temperatura ($\propto T^4$) — con soli 20–40 episodi di training, senza alcuna conoscenza della fisica del sistema, è il risultato più significativo di questa analisi.

6.3.4 Confronto dettagliato

- a. **Modulazione vs commutazione:** PILCO produce un segnale di controllo continuo e modulato, mentre l'isteresi commuta tra 0 % e 100 %. La modulazione continua è superiore sotto diversi aspetti: precisione a regime, assenza di oscillazioni permanenti, minore stress degli attuatori e riduzione del consumo energetico medio.
- b. **Oscillazione permanente:** l'isteresi soffre di un'oscillazione strutturale legata alla banda $\pm\delta$, che non può essere eliminata senza ridurre δ — operazione che a sua volta aumenta la frequenza di commutazione. PILCO non ha questa limitazione.
- c. **Costo di setup:** l'isteresi è operativa immediatamente (non richiede addestramento), mentre PILCO necessita di una fase di training. Questo rende l'isteresi preferibile in scenari “usa e getta” o dove il tempo di commissioning è critico.
- d. **Adattabilità:** PILCO può adattarsi a condizioni operative diverse grazie all'inclusione di variabili di contesto nello stato (Caso 2: T_{amb} , Caso 3: T_{set}). L'isteresi, operando solo sulla misura corrente di T_1 , non ha alcuna consapevolezza delle condizioni ambientali o del setpoint target.

6.4 Limiti riscontrati

- a. **Costo computazionale:** l'ottimizzazione degli iperparametri del GP scala cubicamente con il numero di osservazioni n , con complessità $\mathcal{O}(n^3)$. Per dataset superiori a qualche centinaio di campioni è necessario ricorrere al GP sparso (FITC), già implementato nel codice con $m = 200\text{--}300$ punti induttori.
- b. **Approssimazione gaussiana:** il *moment matching* approssima con una gaussiana distribuzioni che possono essere multimodali, sottostimando l'incertezza nei transitori rapidi. Questo è particolarmente rilevante nei cambi di setpoint del Caso 3, dove la dinamica è fortemente nonlineare.
- c. **Saturazione del controllore:** nei casi di variazione rapida del setpoint, il segnale di controllo Q_1 tende a saturare ai limiti dell'intervallo ammissibile (0 % o 100 %), rallentando il tracking e riducendo il vantaggio di PILCO rispetto all'isteresi durante i transitori.
- d. **Extrapolazione:** il GP fornisce predizioni affidabili solo nell'intervallo dei dati di training. Condizioni operative significativamente al di fuori di questo intervallo (ad esempio T_{set} o T_{amb} molto diversi da quelli visti) producono un aumento dell'incertezza e un potenziale degrado delle prestazioni.
- e. **Correlazione temporale:** PILCO tratta l'incertezza del modello come rumore non correlato nel tempo, il che può portare a una sottostima dell'incertezza cumulata lungo traiettorie lunghe [2].
- f. **Ottimizzazione locale:** l'ottimizzatore L-BFGS trova minimi locali della funzione di costo. La politica finale dipende dall'inizializzazione dei centri e dei pesi della RBF, e non vi è garanzia di ottimalità globale.

Capitolo 7

Conclusioni e Sviluppi Futuri

7.1 Sintesi dei risultati

Questo lavoro ha dimostrato che PILCO è un algoritmo di *policy search* efficace e data-efficient per il controllo termico di precisione sul sistema TCLAB. I risultati principali possono essere così riassunti:

- **Caso 1 — Regolazione a setpoint fisso:** PILCO converge a un controllore stabile in soli 5 rollout di addestramento (più 15 rollout casuali iniziali), raggiungendo un errore a regime contenuto attorno al setpoint di 50 °C. Il segnale di controllo risulta modulato e fisicamente coerente.
- **Caso 2 — Robustezza alla temperatura ambiente:** la politica RBF si adatta automaticamente a temperature ambiente diverse, grazie all'inclusione di T_{amb} nello stato e nella politica. Il GP apprende la dipendenza della dinamica da T_{amb} senza alcuna conoscenza a priori del modello fisico.
- **Caso 3 — Inseguimento del riferimento variabile:** la formulazione basata sull'errore $e = T_1 - T_{\text{set}}$ consente a una singola politica di inseguire setpoint diversi, incluse transizioni di salita e discesa. L'inclusione di episodi con $T_{\text{set}} < T_1^{\text{init}}$ nel training permette al GP di modellare il raffreddamento passivo.
- **Confronto con isteresi:** PILCO offre un controllo significativamente superiore rispetto all'isteresi in termini di precisione a regime (assenza di oscillazione permanente), qualità del segnale di controllo (modulazione continua anziché commutazione on/off) e adattabilità a condizioni operative variabili.

In tutti i casi, il numero di interazioni richiesto è risultato nell'ordine delle decine di episodi — significativamente inferiore rispetto agli approcci *model-free*, confermando l'ipotesi di *sample efficiency* alla base di PILCO.

La progressione tra i tre casi ha inoltre dimostrato un'importante proprietà del framework: la capacità di estendere il controllore a scenari più complessi semplicemente aggiungendo variabili allo stato, senza modificare l'algoritmo core.

7.2 Sviluppi futuri

A partire dai risultati ottenuti e dai limiti identificati, si individuano i seguenti possibili sviluppi:

1. **Validazione su hardware reale:** l'implementazione attuale utilizza un simulatore ODE del TCLAB. Il passo naturale successivo è la validazione sulla piattaforma fisica, dove il rumore dei sensori, i ritardi di comunicazione e le imprecisioni del modello ODE introdurrebbero sfide aggiuntive che metterebbero alla prova la robustezza del GP.
2. **Deep PILCO:** sostituire i GP con reti neurali bayesiane (BNN) o *deep ensembles* per scalare a spazi degli stati ad alta dimensionalità mantenendo una stima dell'incertezza [15]. Questo supererebbe il limite di complessità $\mathcal{O}(n^3)$ del GP standard.
3. **Confronto con PID e MPC:** estendere il confronto attuale (limitato all'isteresi) a controllori PID auto-tarati e a un MPC stocastico che utilizzi il GP appreso da PILCO come modello interno, combinando la *sample efficiency* del GP con le garanzie di stabilità dell'MPC.
4. **Controllo multivariabile:** controllare simultaneamente T_1 e T_2 verso setpoint distinti con due azioni $[Q_1, Q_2]$, sfruttando la struttura MIMO già presente nell'hardware del TCLAB. Questo richiederebbe l'estensione dello spazio delle azioni e della matrice dei pesi W nella funzione di costo.
5. **Vincoli di sicurezza:** integrare vincoli espliciti sulla temperatura massima ammissibile e sulla variazione dell'azione (*safe PILCO*), utilizzando funzioni barriera o formulazioni con vincoli probabilistici per garantire il rispetto dei limiti operativi durante l'apprendimento.
6. **GP sparso avanzato:** adottare approssimazioni sparse più moderne (VFE, SVGP) per ridurre ulteriormente il costo computazionale e abilitare esperimenti di durata maggiore con dataset più ricchi.

Bibliografia

- [1] R. S. Sutton e A. G. Barto, *Reinforcement Learning: An Introduction*, 2^a ed. The MIT Press, 2018.
- [2] M. P. Deisenroth e C. E. Rasmussen, «PILCO: A Model-Based and Data-Efficient Approach to Policy Search,» *Proceedings of the 28th International Conference on Machine Learning*, pp. 465–472, 2011.
- [3] O. Dogru, J. Xie, O. Prakash et al., «Reinforcement Learning in Process Industries: Review and Perspective,» *IEEE/CAA Journal of Automatica Sinica*, vol. 11, n. 2, pp. 283–300, 2024. DOI: 10.1109/JAS.2024.124227
- [4] P. Morcillo-Pallarés et al., «Where Reinforcement Learning Meets Process Control: Review and Guidelines,» *Processes*, vol. 10, n. 11, p. 2311, 2022. DOI: 10.3390/pr10112311
- [5] J. H. Lee et al., «Reinforcement Learning for Process Control: Review and Benchmark Problems,» *International Journal of Control, Automation, and Systems*, 2025. DOI: 10.1007/s12555-024-0990-1
- [6] T. M. Moerland, J. Broekens, A. Plaat e C. M. Jonker, «Model-based Reinforcement Learning: A Survey,» *Foundations and Trends in Machine Learning*, vol. 16, n. 1, pp. 1–118, 2023. DOI: 10.1561/22000000086
- [7] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 1994, ISBN: 978-0-471-61977-2.
- [8] C. J. C. H. Watkins e P. Dayan, «Q-learning,» *Machine Learning*, vol. 8, pp. 279–292, 1992.
- [9] T. P. Lillicrap, J. J. Hunt, A. Pritzel et al., «Continuous Control with Deep Reinforcement Learning,» *International Conference on Learning Representations (ICLR)*, 2016.
- [10] T. Haarnoja, A. Zhou, P. Abbeel e S. Levine, «Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor,» in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, pp. 1861–1870.

- [11] J. G. Schneider, «Exploiting Model Uncertainty Estimates for Safe Dynamic Control Learning,» *Advances in Neural Information Processing Systems*, 1997.
- [12] C. E. Rasmussen e C. K. I. Williams, *Gaussian Processes for Machine Learning*. The MIT Press, 2006, ISBN: 978-0-262-18253-9.
- [13] J. D. Hedengren, *TCLab: Temperature Control Laboratory*, <http://apmonitor.com/pdc/index.php/Main/ArduinoTemperatureControl>, 2019.
- [14] M. Rampazzo, *Modelling a Thermal System Using Physics-Based Methods*, Dispense del corso di Advanced Control Systems, Università degli Studi di Padova, 2024.
- [15] Y. Gal e Z. Ghahramani, «Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning,» in *Proceedings of the 33rd International Conference on Machine Learning*, 2016, pp. 1050–1059.